

RECETAS NÚMERICAS EN C

EL ARTE DE LA COMPUTACIÓN CIENTÍFICA
*traducido por **natits***

Índice general

1. Preliminares	5
1.1. Introducción	5
1.1.1. Entorno computacional y validación de programas	6
1.1.2. Compatibilidad con la Primera Edición	6
1.1.3. Acerca de las referencias	7
1.2. Organización de programas y estructuras de control	8
1.2.1. Estructuras de control	9
1.3. Algunas convenciones de C para la computación científica	10
1.4. Error, precisión y estabilidad	11
2. Solución de ecuaciones algebraicas lineales	13
2.1. Introducción	13
2.2. Conjuntos de ecuaciones no singulares versus singulares	13
2.3. Tareas del álgebra lineal computacional	14
2.4. Matrices de Vandermonde y Matrices de Toeplitz	16
2.4.1. Matrices de Vandermonde	16
2.4.2. Matrices de Toeplitz	19
3. Interpolación y extrapolación	25
3.1. Introducción	25
3.2. Interpolación y Extrapolación Polinomial	27
3.3. Interpolación y Extrapolación con Funciones Racionales	29
3.4. Interpolación por Splines Cúbicos	31
3.5. Cómo buscar en una tabla ordenada	34
3.5.1. Búsqueda con valores correlacionados	35
3.5.2. Coeficientes del Polinomio Interpolador	36
3.5.3. Otro Método	37
3.5.4. Interpolación en dos o más dimensiones	38
3.5.5. Orden superior para mayor precisión	39
3.5.6. Orden Superior para Suavidad: Interpolación Bicúbica	39
3.5.7. Mayor orden para suavidad: Spline Bicúbico	42
4. Modelado de Datos	43
4.1. Introducción	43
4.2. Mínimos Cuadrados como un Estimador de Máxima Verosimilitud	44
4.2.1. Ajuste por Chi-Cuadrada	45
4.3. Mínimos Cuadrados Lineales Generales	47
4.3.1. Solución mediante el uso de las Ecuaciones Normales	48
4.3.2. Solución mediante el uso de la descomposición en valores singulares	50

Capítulo 1

Preliminares

1.1. Introducción

Este libro, al igual que su predecesor, tiene como objetivo enseñarte métodos de computación numérica que sean prácticos, eficientes y, en la medida de lo posible, elegantes. Suponemos a lo largo de este libro que tú, el lector, tienes tareas específicas que quieres resolver. Nuestro trabajo es educarte sobre cómo proceder. A veces podríamos redirigirte brevemente hacia un sendero particularmente bello pero poco útil; sin embargo, en general, te guiaremos por caminos principales que te lleven a destinos prácticos. A lo largo del libro encontrarás opiniones firmes: te diremos qué deberías hacer y qué no. Este tono prescriptivo refleja decisiones conscientes de nuestra parte, y esperamos que no lo encuentres molesto. No afirmamos que nuestros consejos sean infalibles; más bien, respondemos a una tendencia común en la literatura académica de computación, que suele presentar cada método posible sin ofrecer juicios prácticos sobre su mérito relativo. Nosotros, por el contrario, sí emitimos juicios cuando podemos. Con la experiencia, tú mismo formarás tu propia opinión sobre qué tan confiables son nuestros consejos.

Suponemos también que sabes programar en C, ya que ese es el lenguaje de esta versión de *Numerical Recipes* (Segunda Edición). Existe también una versión separada en FORTRAN (Segunda Edición), si prefieres programar en ese lenguaje. Versiones anteriores de *Numerical Recipes* en Pascal y ejemplos en BASIC también están disponibles; aunque no contienen el material adicional de la segunda edición en C y FORTRAN, siguen siendo útiles si tu lenguaje de preferencia es Pascal o BASIC.

Cuando incluimos programas en el texto, se ven así:

Listing 1.1: Cálculo de las fases lunares

```
1 #include <math.h>
2 #define RAD (3.14159265/180.0)
3
4 void flmoon(int n, int np, long *jd, float *frac)
5 /*
6  Nuestros programas comienzan con un comentario introductorio que resume su propósito
7  y explica cómo debe llamarse. Esta rutina calcula las fases de la luna. Dados un entero
8  n y un código np para la fase deseada (np = 0 para luna nueva, 1 para cuarto creciente,
9  2 para luna llena, 3 para cuarto menguante), la rutina devuelve el número juliano jd y
10 la fracción del día frac que se le suma, correspondiente a la enésima aparición de dicha
11 fase desde enero de 1900. Se asume la hora media de Greenwich.
12 */
13 {
14     void nrerror(char error_text[]);
15     int i;
16     float am, as, c, t, t2, extra;
17
18     c = n + np/4.0;          // Así es como comentamos una línea individual.
19     t = c / 1236.85;
20     t2 = t * t;
21     as = 359.2242 + 29.105356 * c;    // No estás realmente obligado a entender
22     am = 306.0253 + 385.816918 * c    // este algoritmo, ¡pero funciona!
23         + 0.010730 * t2;
24     *jd = 2415020 + 28 * n + 7 * np;
25     extra = 0.75933 + 1.53058868 * c + ((1.178e-4) - (1.55e-7) * t) * t2;
26 }
```

```

27  if (nph == 0 || nph == 2)
28      xtra += (0.1734 - 3.93e-4 * t) * sin(RAD * as) - 0.4068 * sin(RAD * am);
29  else if (nph == 1 || nph == 3)
30      xtra += (0.1721 - 4.0e-4 * t) * sin(RAD * as) - 0.6280 * sin(RAD * am);
31  else nerror("nph is unknown in flmoon"); // Así es como indicamos condiciones de
      error.
32
33  i = (int)(xtra >= 0.0 ? floor(xtra) : ceil(xtra - 1.0));
34  *jd += i;
35  *frac = xtra - i;
36  }

```

Si la sintaxis de la definición de funciones mostrada arriba te resulta extraña, entonces probablemente estés acostumbrado a la sintaxis antigua de Kernighan y Ritchie ("K&R"), en lugar de la más reciente ANSI C. En esta edición adoptamos ANSI C como nuestro estándar. Podrías revisar la Sección §1.2, donde se discuten con más detalle los prototipos de funciones en ANSI C.

Observa nuestra convención para manejar todos los errores y casos excepcionales con una instrucción como `nerror("some error message");`. La función `nerror()` forma parte de un pequeño archivo de utilidades, `nrutil.c`, listado en el Apéndice B al final del libro. Este apéndice incluye varias otras utilidades que describiremos más adelante en este capítulo. La función `nerror()` imprime el mensaje de error indicado en tu dispositivo `stderr` (usualmente la pantalla del terminal), y luego invoca la función `exit()`, que termina la ejecución. La función `exit()` está en todas las bibliotecas de C que conocemos; pero si la encuentras ausente, puedes modificar `nerror()` para que haga cualquier otra cosa que detenga la ejecución. Por ejemplo, puedes hacer que haga una pausa para recibir entrada desde el teclado, y luego interrumpir la ejecución manualmente. En algunas aplicaciones, querrás modificar `nerror()` para manejar errores de forma más sofisticada, por ejemplo para transferir el control a otro punto, con una bandera de error o un código de error establecido.

Tendremos más que decir sobre el lenguaje de programación C, sus convenciones y estilo, en §1.1 y §1.2.

1.1.1. Entorno computacional y validación de programas

Nuestro objetivo es que los programas de este libro sean lo más portables posible, entre diferentes plataformas (modelos de computadora), diferentes sistemas operativos y diferentes compiladores de C. El lenguaje C fue diseñado con este tipo de portabilidad en mente. Sin embargo, hemos descubierto que no hay sustituto para comprobar todos los programas en una variedad de compiladores, en el proceso revelando diferencias en la estructura o contenido de las bibliotecas, e incluso diferencias ocasionales en la sintaxis permitida. Como sustitutos de la gran cantidad de combinaciones posibles, hemos probado todos los programas de este libro en las combinaciones de máquinas, sistemas operativos y compiladores que se muestran en la tabla adjunta. Más generalmente, los programas deberían ejecutarse sin modificaciones en cualquier compilador que implemente el estándar ANSI C, como se describe por ejemplo en el excelente libro de Harbison y Steele [?]. Con pequeñas modificaciones, nuestros programas deberían ejecutarse en cualquier compilador que implemente el estándar *de facto* K&R [?]. Un ejemplo del tipo de incompatibilidad trivial a tener en cuenta es que ANSI C requiere que las funciones de asignación de memoria `malloc()` y `free()` se declaren mediante la cabecera `stdlib.h`; algunos compiladores más antiguos requieren que se declaren con el archivo de cabecera `malloc.h`, mientras que otros las consideran como funciones inherentes al lenguaje y no requieren cabecera alguna.

Al validar los programas, hemos tomado el código fuente directamente de la versión legible por máquina del manuscrito del libro, para disminuir la posibilidad de errores tipográficos. Los programas "driver" o de demostración que usamos como parte de nuestras validaciones están disponibles por separado como el *Numerical Recipes Example Book (C)*, así como en formato legible por máquina. Si planeas usar más de unos pocos programas de este libro, o si planeas usar programas de este libro en más de una computadora diferente, entonces podrías encontrar útil obtener una copia de estos programas de demostración.

Por supuesto, sería ingenuo afirmar que no hay errores en nuestros programas, y no hacemos tal afirmación. Hemos sido muy cuidadosos y nos hemos beneficiado de la experiencia de muchos lectores que nos han escrito. Si encuentras un nuevo error, ¡por favor documéntalo y háznoslo saber!

1.1.2. Compatibilidad con la Primera Edición

Si estás acostumbrado a las rutinas de *Numerical Recipes* de la Primera Edición, ten la seguridad de que casi todas ellas siguen aquí, con los mismos nombres y funcionalidades, a menudo con mejoras importantes en el propio código. Además, esperamos que pronto te familiarices igualmente con las capacidades añadidas de las más de 100 rutinas que son nuevas en esta edición.

Hemos retirado un pequeño número de rutinas de la Primera Edición, aquellas que creemos que han sido claramente superadas por métodos mejores implementados en esta edición. Una tabla, a continuación, enumera las rutinas retiradas y sugiere reemplazos.

Los usuarios de la Primera Edición también deben saber que algunas rutinas comunes a ambas ediciones han sufrido cambios en sus interfaces de llamada, por lo que no son directamente “compatibles sin modificaciones”. Una lista bastante completa es: `chsone`, `chstwo`, `covsrt`, `dfpmin`, `laguer`, `lfit`, `memcof`, `mrqcof`, `mrqmin`, `pzextr`, `ran4`, `realft`, `rzextr`, `shoot`, `shootf`. Puede que haya otras (dependiendo en parte de qué impresión de la Primera Edición se tome para la comparación). Si has escrito software de complejidad apreciable que depende de rutinas de la Primera Edición, *no* recomendamos reemplazarlas ciegamente por las rutinas correspondientes en este libro. Recomendamos que cualquier nuevo esfuerzo de programación utilice las nuevas rutinas.

1.1.3. Acerca de las referencias

Encontrarás referencias y sugerencias para lecturas adicionales al final de la mayoría de las secciones de este libro. Las referencias se citan en el texto mediante números entre corchetes como este: [3].

Dado que los algoritmos computacionales a menudo circulan de forma informal durante bastante tiempo antes de aparecer en una publicación formal, la tarea de descubrir la “literatura primaria” es a veces bastante difícil. No hemos intentado hacer esto, y no pretendemos alcanzar ningún grado de exhaustividad bibliográfica en este libro. Para temas en los que existe una cantidad considerable de literatura secundaria (discusión en libros de texto, revisiones, etc.), hemos limitado conscientemente nuestras referencias a unas pocas fuentes secundarias útiles, especialmente aquellas con buenas referencias a la literatura primaria. Cuando la literatura secundaria existente es insuficiente, damos referencias a algunas fuentes primarias que están destinadas a servir como punto de partida para lecturas adicionales, no como bibliografías completas del área.

El orden en que se listan las referencias no es necesariamente significativo. Refleja un compromiso entre listar las referencias citadas en el orden en que aparecen y listar sugerencias para lecturas adicionales en un orden aproximadamente priorizado, comenzando con las fuentes más útiles.

Las tres secciones restantes de este capítulo repasan algunos conceptos básicos de programación (estructuras de control, etc.), discuten un conjunto de convenciones específicas de C que hemos adoptado en este libro, e introducen algunos conceptos fundamentales en análisis numérico (error de redondeo, etc.). Luego, entramos en el material sustantivo del libro.

1.2. Organización de programas y estructuras de control

A veces nos gusta señalar las analogías estrechas entre los programas de computadora, por un lado, y la poesía escrita o las partituras musicales, por el otro. Todos ellos se presentan como medios visuales, símbolos en una página bidimensional o en una pantalla de computadora. Sin embargo, en los tres casos, la representación visual bidimensional, *congelada en el tiempo*, comunica (o se supone que comunica) algo bastante diferente: es decir, un proceso que *se despliega en el tiempo*. Un poema está destinado a ser leído; la música, a ser interpretada; un programa, a ser ejecutado como una serie secuencial de instrucciones.

En los tres casos, el objetivo de la comunicación, en su forma visual, es un ser humano. El objetivo es transmitirle, tan eficientemente como sea posible, el mayor grado de comprensión anticipada sobre cómo se desplegará el proceso. En la poesía, este objetivo humano es el lector. En la música, es el intérprete. En la programación, es el usuario del programa.

Ahora bien, podrías objetar que el objetivo de la comunicación de un programa no es un humano sino una computadora, que el usuario del programa es solo un intermediario irrelevante, alguien que alimenta a la máquina. Este es quizá el caso cuando un ejecutivo introduce un disquete en una computadora y carga un programa opaco en formato binario ejecutable. A esa computadora, en ese caso, no le importa si el programa fue escrito con “buenas prácticas de programación” o no.

Sin embargo, nuestra visión para ti, lector de este libro, es bastante diferente. Necesitas saber, o querer saber, no solo *qué* hace un programa, sino también *cómo* lo hace, para que puedas modificarlo y adaptarlo a tu aplicación particular. Necesitas que otros puedan ver lo que has hecho, para que puedan criticarlo o mejorarlo. Por tanto, el objetivo deseado de una buena práctica de programación es que el código sea *mantenible* o *reutilizable*, no el éxito de la ejecución del programa en sí, que seguramente lo es gracias a la máquina, no al humano.

Esto también implica reconocer que la poesía, la música y la programación —las tres construcciones simbólicas del lenguaje— tienen estructuras jerárquicas con muchos niveles diferentes. (Estas jerarquías) van desde unidades pequeñas (morfemas) hasta unidades más grandes. En los programas, estas unidades van desde caracteres hasta líneas; las líneas se agrupan en frases, las frases en sentencias; las sentencias en bloques; y los bloques en módulos o funciones de nivel superior. Las notas musicales forman compases, luego frases, luego secciones, sinfonías, etc.

Entonces las sentencias de programa, como `k=(a[j+1]=b+c/3.0);` pueden escribirse. Aquí, el mejor consejo de programación es simplemente *sé claro*, o (de forma correspondiente) *no seas demasiado astuto*. Podrías estar momentáneamente orgulloso de ti mismo al escribir la línea:

```
1 k=(2-3)+(1+3+4)/2;
```

si quieres permutar cíclicamente uno de los valores `j = (0, 1, 2)` en `k = (1, 2, 0)`. Luego te arrepentirás, cuando intentes entender esa línea. Mejor —y probablemente más rápido— es:

```
1 k+=1;
2 if (k == 3) k=0;
```

Muchos estilistas de programación incluso argumentarían a favor de la versión literalmente detallada:

```
1 switch (j) {
2     case 0: k=1; break;
3     case 1: k=2; break;
4     case 2: k=0; break;
5     default: {
6         fprintf(stderr, "unexpected value for j");
7         exit(1);
8     }
9 }
```

porque es tanto más clara como además protegida contra suposiciones erróneas sobre los posibles valores de `j`. Nuestra preferencia entre las implementaciones es la del medio.

En este ejemplo simple, de hecho hemos recorrido varios niveles de jerarquía: las sentencias frecuentemente vienen en “grupos” o “bloques” que tienen sentido solo cuando se toman como un todo. El fragmento del medio anterior es un ejemplo. Otro es:

```
1 swap=a[i];
2 a[i]=b[j];
3 b[j]=swap;
```


lo cual tiene sentido inmediato para cualquier programador como el intercambio de dos variables, mientras que:

```
1 ans=sum=0.0;  
2 n=1;
```

es muy probable que sea una inicialización de variables antes de un proceso iterativo. Este nivel de jerarquía en un programa suele ser evidente a simple vista. Es una buena práctica de programación agregar comentarios a este nivel, por ejemplo, “inicializar” o “intercambiar variables”.

El siguiente nivel es el de *estructuras de control*. Estas son cosas como la construcción `switch` del ejemplo anterior, bucles `for`, y demás. Este nivel es suficientemente importante, relacionado con la estructura jerárquica del código de este libro, que volveremos a él más adelante en este capítulo.

Aún más alto en la jerarquía, tenemos funciones y módulos, y la visión global de la organización del código. En el análogo musical, estamos ahora en el nivel de movimientos y obras completas. En estos niveles, *modularización* y *encapsulamiento* se convierten en conceptos importantes de programación, el objetivo siendo que los programas interactúen unos con otros solo mediante interfaces claramente definidas. Una buena práctica de modularización es esencial para el éxito de proyectos grandes y complicados, especialmente aquellos que requieren esfuerzos de más de un programador. También es útil (aunque no tan esencial) en las tareas menos masivas de programación que se encuentran en otras partes de este libro.

Algunos lenguajes de computadora, como Modula-2 y C++, promueven una buena modularización en formas que están ausentes en C. En Modula-2, por ejemplo, funciones, definiciones de tipo y estructuras de datos pueden encapsularse en “módulos” que se comunican mediante interfaces públicas declaradas, y cuyos funcionamiento internos están ocultos del resto del programa [?]. En el lenguaje C++, el concepto clave es la *clase*, una generalización definida por el usuario de tipos de datos que proporciona ocultamiento de datos, inicialización automática, manejo de memoria, gestión dinámica de tipos, y sobrecarga de operadores (es decir, la extensión definida por el usuario de operadores como `+` y `-` para que actúen en estructuras definidas por el usuario) [?]. Definir apropiadamente las estructuras de datos que se pasan entre unidades de programa, las clases, puede clarificar y delimitar las interfaces públicas de estas unidades, reduciendo las posibilidades de errores de programación y también permitiendo un grado considerable de verificación tanto en tiempo de compilación como en tiempo de ejecución.

Más allá de la modularización, aunque basándose en ella, están los conceptos de *programación orientada a objetos*. Aquí, un lenguaje de programación como C++ o Turbo Pascal 5.5 [?] permite que la interfaz pública de un módulo acepte redefiniciones de tipos o acciones, y que estas redefiniciones se compartan desde el nivel más alto del módulo hasta el más bajo (lo que se llama *polimorfismo*). Por ejemplo, una rutina escrita para invertir una matriz de números reales podría —dinámicamente, en tiempo de ejecución— manejar matrices complejas también, usando tipos de datos con componentes reales e imaginarios y sus definiciones correspondientes.

Conceptos adicionales como *herencia* (la habilidad de definir un tipo de datos que “hereda” toda la estructura de otro tipo de datos, más funcionalidad adicional), y *extensibilidad de objetos* (la habilidad de añadir funcionalidad a un módulo sin acceso a su código fuente, por ejemplo, en tiempo de ejecución), también entran en juego.

Aunque no pretendemos introducirte en los objetivos más amplios de la programación modular y orientada a objetos, los fragmentos de código de este libro no son módulos, por al menos dos razones. Primero, el lenguaje C realmente no realiza este tipo de encapsulamiento. Segundo, como sabes, el lector necesita ver cómo se implementan cosas como funciones, no solo una declaración estándar o un encabezado. Si el lector no puede ver la implementación, inevitablemente se perdería el control de alto nivel del flujo del programa. Por tanto, mostramos fragmentos funcionales completos, no módulos encapsulados o definidos por clases.

1.2.1. Estructuras de control

La ejecución de un programa ocurre en el tiempo, pero no estrictamente en el orden lineal en que las sentencias están escritas. Las sentencias de programa que afectan el orden en que se ejecutan otras sentencias, o que afectan si se ejecutan o no, se llaman *sentencias de control*. Las sentencias de control nunca tienen mucho sentido por sí solas. Solo tienen sentido en el contexto de los grupos o bloques de sentencias que a su vez controlan. Si piensas en esos bloques como párrafos que contienen oraciones, entonces las sentencias de control pueden pensarse como la sangría del párrafo y la puntuación entre oraciones, no como las palabras dentro de las oraciones.

Ahora podemos decir cuál es el objetivo de la programación estructurada. Es: *hacer que el control del programa sea manifestamente visible en la presentación visual del programa*. Puedes ver que este objetivo

no tiene nada que ver con cómo la computadora ve el programa. Como ya se dijo, a las computadoras no les importa si usas programación estructurada o no. Sin embargo, a los lectores humanos *sí* les importa. A ti mismo también te importará, una vez que descubras lo mucho más fácil que es perfeccionar y depurar un programa bien estructurado que uno cuya estructura de control es oscura.

Logras el objetivo de la programación estructurada de dos maneras complementarias. Primero, te familiarizas con el pequeño número de estructuras de control esenciales que aparecen una y otra vez en programación, y que por tanto tienen representaciones convenientes en la mayoría de los lenguajes de programación. Deberías aprender a pensar tus tareas de programación, en la medida de lo posible, exclusivamente en términos de estas estructuras de control estándar. Al escribir programas, deberías adquirir el hábito de representar esas estructuras estándar de forma coherente y convencional.

“¿Esto no inhibe la creatividad?”, preguntan a veces nuestros estudiantes. Sí, igual que la creatividad de Mozart fue inhibida por la forma sonata, o la de Shakespeare por los requerimientos métricos del soneto. El punto es que la creatividad, cuando se pretende comunicar, ama vivir bajo las inhibiciones de restricciones apropiadas de formato.

Segundo, deberías evitar, en la medida de lo posible, sentencias de control que hagan difícil ver el flujo del programa de un vistazo. Esto significa, en la práctica, que *deberías evitar las sentencias con saltos y las instrucciones goto*. Es el *goto* lo que interrumpe el flujo (aunque también interrumpa la lectura de un programa); la sentencia nombrada también lo hace, pero en menor grado. De hecho, cada vez que encuentres una sentencia nombrada en un programa, pronto desarrollarás una sensación de desagrado y una pregunta en tu mente: *¿por qué está esto aquí?* Porque el flujo se está rompiendo.

Entonces, ¿de dónde viene el control del flujo en una rama a esta línea, y bajo qué circunstancias esa interrupción del flujo era inevitable?

1.3. Algunas convenciones de C para la computación científica

El lenguaje C fue diseñado originalmente para programación de sistemas, no para computación científica. En comparación con otros lenguajes de programación de alto nivel, C sitúa al programador “muy cerca de la máquina” en varios aspectos. Es rico en operadores, dando acceso directo a la mayoría de las capacidades de un conjunto de instrucciones en lenguaje ensamblador. Tiene una gran variedad de tipos de datos intrínsecos (enteros cortos y largos, con y sin signo; números reales en precisión simple y doble; tipos puntero; etc.) y una sintaxis concisa para efectuar conversiones e indirectas. Define una aritmética sobre punteros (direcciones) que se adapta elegantemente al direccionamiento por arreglos y es altamente compatible con la estructura de registros de índice de muchas computadoras.

La portabilidad siempre ha sido otro punto fuerte del lenguaje C. C es el lenguaje subyacente del sistema operativo UNIX; tanto el lenguaje como el sistema operativo han sido ahora implementados en literalmente cientos de arquitecturas diferentes. La universalidad, portabilidad y flexibilidad del lenguaje han atraído a números crecientes de científicos e ingenieros. Es comúnmente usado para controlar instrumentos científicos y hardware en tiempo real, a menudo en parte debido al hecho de que el núcleo estándar de UNIX es menos costoso que un sistema operativo completo para este propósito.

El uso de C para cálculos científicos de alto nivel como análisis de datos, modelado y trabajo numérico en punto flotante ha evolucionado de forma más lenta. Esto se debe en parte a la posición histórica favorecida del FORTRAN como la lengua materna de casi todos los científicos e ingenieros nacidos antes de 1960, y la mayoría de los nacidos poco después. En parte, también, la lentitud con la que la generación de científicos formada en C ha ingresado a la computación científica se debe a deficiencias en el lenguaje que los científicos computacionales han sido (creemos, tercamente) lentos en reconocer. Ejemplos son la falta de una forma adecuada para elevar números a potencias enteras pequeñas, y la “conversión implícita de `float` a `double`”, que se discute más adelante. Muchas, aunque no todas, estas deficiencias han sido corregidas en la norma ANSI C. Algunas deficiencias restantes sin duda desaparecerán con el tiempo.

Otra inhibición notable para la conversión masiva de científicos al culto de C ha sido, hasta el momento de escribir este texto, la falta de una biblioteca bien desarrollada de rutinas científicas o numéricas. Esta es la carencia que pretendemos abordar en esta edición de *Numerical Recipes*. Ciertamente no pretendemos ser una solución completa al problema. Esperamos inspirar otros esfuerzos, y presentar mediante el ejemplo un conjunto sensato y práctico de convenciones para la programación científica en C.

La necesidad de convenciones de programación en C es muy grande. Lejos de ser un problema de limitación impuesto por el lenguaje (nuestra experiencia repetida con Pascal lo prueba), el problema en C es escoger las mejores y más naturales técnicas entre múltiples posibilidades, y luego usar esas técnicas de forma completamente

coherente y clara en todo el programa. En el resto de esta sección, exponemos algunas de estas técnicas, y describimos las convenciones adoptadas que se usan en todas las rutinas de este libro.

1.4. Error, precisión y estabilidad

Aunque no asumimos entrenamiento previo del lector en análisis numérico formal, necesitaremos asumir una comprensión común de algunos conceptos clave. Los definiremos brevemente en esta sección.

Las computadoras almacenan números no con precisión infinita, sino más bien en una aproximación que puede empacarse en un número fijo de *bits* (dígitos binarios) o *bytes* (grupos de 8 bits). Casi todas las computadoras permiten al programador elegir entre varias representaciones o tipos de datos. Los tipos de datos pueden diferir en el número de bits utilizados (el *wordlength*), pero también en el aspecto más fundamental de si el número almacenado se representa en formato *entero fijo* (*int* o *long*) o *punto flotante* (*float* o *double*).

Un número en representación entera es exacto. La aritmética entre números en representación entera también es exacta, con la salvedad de que (i) el resultado no esté fuera del rango de enteros (usualmente con signo) que pueden representarse, y (ii) que la división se interprete como produciendo un resultado entero, descartando cualquier residuo entero.

En la representación de punto flotante, un número se representa internamente mediante un bit de signo s (interpretado como positivo o negativo), un exponente entero exacto E , y una mantisa entera exacta M . Juntos representan el número:

$$s \times M \times B^{-E} \quad (1.3.1)$$

donde B es la base de la representación (usualmente $B = 2$, pero a veces $B = 16$), y E es el sesgo del exponente, una constante fija para cualquier máquina y representación dada. Un ejemplo se muestra en la Figura 1.3.1.

Varios patrones de bits de punto flotante pueden representar el mismo número. Si $B = 2$, por ejemplo, una mantisa con ceros en los bits más significativos puede desplazarse a la izquierda, es decir, multiplicarse por una potencia de 2, si el exponente se incrementa por una cantidad compensatoria. Pero los patrones de bits que están “tan desplazados a la izquierda como sea posible” se denominan *normalizados*. La mayoría de las computadoras producen resultados normalizados, ya que entonces no se desperdician bits de mantisa, y permiten un mayor rango de precisión expresable. Esta normalización es parte de la especificación precisa de la representación.

La aritmética entre números en representación de punto flotante no es exacta, incluso si los operandos están exactamente representados (es decir, tienen valores exactos en la forma de la ecuación 1.3.1). Por ejemplo, dos números de punto flotante se suman primero desplazando a la derecha (dividiendo entre dos) la mantisa del operando menor (en magnitud), incrementando simultáneamente su exponente, hasta que ambos operandos tengan el mismo exponente. Los bits de orden inferior (menos significativos) del operando más pequeño se pierden por este desplazamiento. Si los dos operandos difieren demasiado en magnitud, entonces el operando más pequeño es reemplazado efectivamente por cero, ya que es desplazado a la derecha hasta desaparecer.

El número de punto flotante más pequeño (en magnitud) que, al ser sumado al número 1.0, produce un resultado de punto flotante distinto de 1.0, se denomina *precisión de máquina* ϵ_m . Una computadora típica con $B = 2$ y una palabra de 32 bits tiene ϵ_m alrededor de 3×10^{-8} . (Una discusión más detallada de las características de la máquina, y un programa para determinarlas, se da en la §20.1.)

En términos generales, la precisión de máquina ϵ_m es el error fraccional en el cual los números de punto flotante pueden ser representados, correspondiente al bit de mayor orden de la mantisa. Prácticamente cualquier operación aritmética entre números de punto flotante debe considerarse como introduciendo un error fraccional adicional de al menos ϵ_m . Este tipo de error se llama *error de redondeo*.

Es importante entender que ϵ_m no es el número de punto flotante más pequeño que puede representarse en una máquina. Ese número depende de cuántos bits hay en el exponente, mientras que ϵ_m depende de cuántos bits hay en la mantisa.

Los errores de redondeo se acumulan con el número de operaciones de cálculo. Si, durante el cálculo de un valor, realizas N operaciones aritméticas, podrías tener la suerte de que el error total por redondeo sea del orden de $\sqrt{N}\epsilon_m$, si los errores de redondeo ocurren al azar hacia arriba o hacia abajo. (Esta estimación proviene de una caminata aleatoria). Sin embargo, esta estimación puede fallar de forma grave por dos razones:

1. Muy frecuentemente sucede que las regularidades de tu cálculo, o las peculiaridades de tu computadora, hacen que los errores de redondeo se acumulen preferentemente en una dirección. En este caso, el error total será del orden de $N\epsilon_m$.

2. Algunas situaciones especialmente desfavorables pueden aumentar enormemente el error de redondeo de operaciones individuales. Generalmente, esto puede rastrearse hasta la sustracción de dos números casi iguales, dando un resultado cuyos bits significativos son esos pocos bits de menor orden en los que difieren. Podrías pensar que tal “sustracción coincidente” es poco probable. No siempre es así. Algunas expresiones matemáticas aumentan enormemente su probabilidad de ocurrencia. Por ejemplo, en la fórmula conocida para la solución de una ecuación cuadrática,

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{1.3.2}$$

la suma o resta se vuelve delicada y propensa al error por redondeo siempre que $ac \ll b^2$. (En la §5.6 aprenderemos cómo evitar este problema en particular).

Capítulo 2

Solución de ecuaciones algebraicas lineales

2.1. Introducción

Un sistema de ecuaciones algebraicas lineales luce así:

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1N}x_N &= b_1 \\a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2N}x_N &= b_2 \\a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + \cdots + a_{3N}x_N &= b_3 \\&\vdots \\a_{M1}x_1 + a_{M2}x_2 + a_{M3}x_3 + \cdots + a_{MN}x_N &= b_M\end{aligned}\tag{2.0.1}$$

Aquí las N incógnitas x_j , $j = 1, 2, \dots, N$ están relacionadas por M ecuaciones. Los coeficientes a_{ij} con $i = 1, 2, \dots, M$ y $j = 1, 2, \dots, N$ son números conocidos, al igual que los valores del lado derecho b_i , $i = 1, 2, \dots, M$.

2.2. Conjuntos de ecuaciones no singulares versus singulares

Si $N = M$, entonces hay tantas ecuaciones como incógnitas, y hay una buena posibilidad de resolver un conjunto único de soluciones para los x_j . Analíticamente, puede no haber una solución única si una o más de las M ecuaciones son combinaciones lineales de las otras, una condición llamada *degeneración por filas*, o si todas las ecuaciones contienen ciertas variables únicamente en exactamente la misma combinación lineal, llamada *degeneración por columnas*. (Para matrices cuadradas, una degeneración por filas implica una por columnas, y viceversa.) Un conjunto de ecuaciones que es degenerado se llama *singular*. Consideraremos matrices singulares en más detalle en la §2.6.

Numéricamente, al menos dos cosas adicionales pueden salir mal:

- Aunque no sean combinaciones lineales exactas entre sí, algunas de las ecuaciones pueden ser tan cercanas a ser linealmente dependientes que los errores de redondeo en la máquina las hacen depender linealmente en alguna etapa del proceso de solución. En este caso, tu procedimiento numérico fallará, y puede indicarte que ha fallado.
- Errores de redondeo acumulados durante el proceso de solución pueden ocultar la verdadera solución. Este problema ocurre especialmente si N es muy grande. El procedimiento numérico puede no fallar algorítmicamente. Sin embargo, devuelve un conjunto de x 's que son erróneos, como puede descubrirse al sustituir directamente de nuevo en las ecuaciones originales. Cuanto más cercano esté un conjunto de ecuaciones a ser singular, más probable es que esto suceda, ya que se producirán cancelaciones cada vez más precisas durante la solución. De hecho, el ítem anterior puede verse como el caso especial donde la pérdida de significado es puramente total.

Gran parte de la sofisticación de los “paquetes de solución de ecuaciones lineales” complicados está dedicada a detectar y/o corregir estas dos patologías. A medida que trabajas con conjuntos grandes de ecuaciones, desarrollarás un sentido de cuándo se requiere tal sofisticación. Es difícil dar directrices firmes, ya que no existe

tal cosa como un “problema lineal típico”. Pero aquí hay una regla general aproximada: conjuntos lineales con N tan grandes como 20 o 50 pueden resolverse rutinariamente en precisión simple (representaciones en punto flotante de 32 bits), sin recurrir a métodos sofisticados, si las ecuaciones no son demasiado singulares. Con precisión doble (60 o 64 bits), este número puede razonablemente extenderse hasta N tan grande como varios cientos, antes de que la precisión de la máquina imponga el límite, no el tiempo de cómputo.

Incluso conjuntos más grandes, N en los miles o más, pueden resolverse cuando los coeficientes son dispersos (es decir, en su mayoría cero), mediante métodos que aprovechan la estructura de dispersión. Discutimos esto con más detalle en la §2.7.

En el otro extremo del espectro, uno a veces encuentra problemas lineales donde, aunque el problema subyacente no es singular, está cerca de serlo. En este caso, puedes necesitar recurrir a métodos sofisticados incluso para el caso de $N = 10$ (aunque rara vez para $N = 5$). La descomposición en valores singulares (§2.6) es una técnica que puede convertir problemas singulares en no singulares, en los cuales el caso adicional de cercanía a la singularidad se vuelve innecesario.

2.3. Tareas del álgebra lineal computacional

Consideraremos las siguientes tareas como parte del enfoque general de este capítulo:

- Solución de la ecuación matricial $A \cdot x = b$ para un vector incógnita x , donde A es una matriz cuadrada de coeficientes, el punto denota multiplicación matricial, y b es un vector conocido del lado derecho (§2.1–§2.10).
- Solución de más de una ecuación matricial $A \cdot x_j = b_j$ para un conjunto de vectores x_j , $j = 1, 2, \dots$, cada uno correspondiente a un diferente vector conocido del lado derecho b . En esta tarea, la clave es que la matriz A se mantiene constante, mientras que los lados derechos, los b 's, cambian (§2.1–§2.10).
- Cálculo de la matriz A^{-1} , que es la inversa de una matriz cuadrada A , es decir, $A \cdot A^{-1} = A^{-1} \cdot A = I$, donde I es la matriz identidad (todas las diagonales en uno, el resto en cero). Esta tarea es equivalente, para una matriz $N \times N$, a la tarea anterior con N vectores b_j ($j = 1, 2, \dots, N$), que son los vectores unitarios (los b 's tienen ceros excepto por un 1 en la componente j -ésima). Los x 's correspondientes son entonces las columnas de la matriz inversa de A (§2.1 y §2.3).
- Cálculo del determinante de una matriz cuadrada A (§2.3).

Si $M < N$, o si $M = N$ pero las ecuaciones son degeneradas, entonces efectivamente hay menos ecuaciones independientes que incógnitas. En este caso puede no haber solución, o puede haber más de una solución para el vector x . En el segundo caso, el espacio de soluciones consiste en una solución particular x_0 , sumada a cualquier combinación lineal de (típicamente) $N - M$ vectores (los cuales se dice que pertenecen al núcleo de la matriz A). La tarea de encontrar el espacio de soluciones de A involucra:

- Descomposición en valores singulares de una matriz A .

Este tema se trata en la §2.6.

En el caso opuesto, donde hay más ecuaciones que incógnitas, $M > N$. Cuando esto ocurre, en general no hay solución exacta x para la ecuación (2.0.1), y el conjunto de ecuaciones se dice que está *sobredeterminado*. Sin embargo, sucede con frecuencia que la “mejor” solución de compromiso es aquella que más se aproxima a satisfacer todas las ecuaciones simultáneamente. La proximidad se define en el sentido de mínimos cuadrados, es decir, la suma de los cuadrados de las diferencias entre los lados izquierdo y derecho de la ecuación (2.0.1) se minimiza. Entonces, el problema sobredeterminado lineal se reduce a un problema de mínimos cuadrados, que se llama:

- Problema de mínimos cuadrados lineales.

Las ecuaciones reducidas a resolver pueden escribirse como el conjunto $N \times N$ de ecuaciones:

$$A^T \cdot A \cdot x = A^T \cdot b \quad (2.0.4)$$

donde A^T denota la traspuesta de la matriz A . Las ecuaciones (2.0.4) se llaman las *ecuaciones normales* del problema de mínimos cuadrados lineales. Existe una conexión estrecha entre la descomposición en valores singulares y el problema de mínimos cuadrados lineales, y este último también se discute en la §2.6. Debes tener en cuenta que la solución directa de las ecuaciones normales (2.0.4) no es, en general, la mejor manera de encontrar soluciones de mínimos cuadrados.

Otros temas de este capítulo incluyen:

- Mejora iterativa de una solución (§2.5)
- Diversas formas especiales: simétrica definida positiva (§2.9), tridiagonal (§2.4), diagonal por bandas (§2.4), Toeplitz (§2.8), Vandermonde (§2.8), dispersa (§2.7)
- “Inversión rápida de matrices” de Strassen (§2.11).

2.4. Matrices de Vandermonde y Matrices de Toeplitz

En la Sección §2.4 se trató especialmente el caso de una matriz tridiagonal, porque ese tipo particular de sistema lineal admite una solución en solo orden N operaciones, en lugar de orden N^3 como en el problema lineal general. Cuando existen tales tipos particulares, es importante conocerlos. Tus ahorros computacionales, en caso de que alguna vez te toque trabajar en un problema que involucre el tipo correcto de matriz, pueden ser enormes.

Esta sección trata dos tipos especiales de matrices que pueden resolverse en orden N^2 operaciones, no tan buenas como las tridiagonales, pero mucho mejores que el caso general. (Aparte del conteo de operaciones, estos dos tipos no tienen nada en común.) Las matrices del primer tipo, llamadas *matrices de Vandermonde*, aparecen en algunos problemas relacionados con el ajuste de polinomios, la reconstrucción de distribuciones a partir de sus momentos, y otros contextos. En este libro, por ejemplo, un problema de Vandermonde aparece en la §3.5. Las matrices del segundo tipo, llamadas *matrices de Toeplitz*, tienden a aparecer en problemas que involucran deconvolución y procesamiento de señales. En este libro, un problema de Toeplitz se encuentra en la §13.7.

Estas no son las *únicas* matrices especiales que vale la pena conocer. Las *matrices de Hilbert*, cuyos componentes son de la forma $a_{ij} = 1/(i + j - 1)$, $i, j = 1, \dots, N$, pueden invertirse mediante un algoritmo exacto entero, y son *muy difíciles* de invertir de cualquier otra forma, ya que son notoriamente mal condicionadas (ver [1] para más detalles). Las fórmulas de Sherman-Morrison y Woodbury, discutidas en la §2.7, a veces pueden usarse para convertir algunos sistemas nuevos en otros viejos. La referencia [2] da otros nombres de matrices especiales. El lector encontrará que los nombres de matrices tienden a aparecer con frecuencia.

2.4.1. Matrices de Vandermonde

Una matriz de Vandermonde de tamaño $N \times N$ está completamente determinada por N números arbitrarios $x_1, x_2, \dots, x_N$, en términos de los cuales sus N^2 componentes son las potencias enteras x_i^{j-1} , $i, j = 1, \dots, N$. Evidentemente hay dos formas posibles, dependiendo de si vemos los i como filas, j como columnas, o viceversa. En el primer caso, obtenemos un sistema lineal de ecuaciones que luce así:

$$\begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{N-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{N-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^{N-1} \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_N \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \quad (2.8.1)$$

Realizando la multiplicación matricial, se verá que esta ecuación resuelve los coeficientes desconocidos c_i , que ajustan un polinomio a los N pares de abscisas y ordenadas (x_j, y_j) . Justamente este problema aparecerá en la sección §3.5, y la rutina dada allí resolverá (2.8.1) mediante el método que estamos por describir.

La identificación alternativa de filas y columnas conduce al conjunto de ecuaciones

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_1 & x_2 & \cdots & x_N \\ x_1^2 & x_2^2 & \cdots & x_N^2 \\ \vdots & \vdots & & \vdots \\ x_1^{N-1} & x_2^{N-1} & \cdots & x_N^{N-1} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix} = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_N \end{bmatrix} \quad (2.8.2)$$

Escriba esto y verá que también está relacionado con el *problema de los momentos*: Dado el valor de N puntos x_i , los pesos desconocidos w_i se asignan para que igualen los valores dados q_j de los primeros N momentos. (Para más sobre este problema, consulte [3].) La rutina que se da en esta sección resuelve (2.8.2).

El método de solución de ambas ecuaciones (2.8.1) y (2.8.2) está estrechamente relacionado con la fórmula de interpolación polinomial de Lagrange, que formalmente no trataremos hasta §3.1. No obstante, la siguiente derivación debería ser comprensible:

Sea $P_j(x)$ el polinomio de grado $N - 1$ definido por

$$P_j(x) = \prod_{\substack{i=1 \\ i \neq j}}^N \frac{x - x_i}{x_j - x_i} = \sum_{k=1}^N A_{jk} x^{k-1} \quad (2.8.3)$$

Aquí la igualdad del lado derecho define los componentes de la matriz A_{jk} como los coeficientes que surgen cuando se multiplica el producto y se recogen términos semejantes.

El polinomio $P_j(x)$ es una función de x en general. Pero se notará que está diseñado cuidadosamente para tomar el valor cero en todos x_i con $i \neq j$, y tener valor unidad en $x = x_j$. En otras palabras,

$$P_j(x_i) = \delta_{ij} = \sum_{k=1}^N A_{jk} x_i^{k-1} \quad (2.8.4)$$

Pero (2.8.4) dice que A_{jk} es exactamente la inversa de la matriz de componentes x_i^{k-1} , que aparece en (2.8.2), con el subíndice como índice de columna. Por tanto, la solución de (2.8.2) usando esa matriz inversa da el siguiente lado derecho,

$$w_j = \sum_{k=1}^N A_{jk} q_k \quad (2.8.5)$$

Para el sistema transpuesto (2.8.1), podemos usar el hecho de que la inversa de la transpuesta es la transpuesta de la inversa, así

$$c_j = \sum_{k=1}^N A_{kj} y_k \quad (2.8.6)$$

La rutina en la §3.5 implementa esto.

Resta encontrar una buena manera de multiplicar los términos monomiales en (2.8.3), para obtener los componentes de A_{jk} . Esto es esencialmente un problema de contabilidad, y te permitiremos leer la rutina tú mismo para ver cómo puede resolverse. Un truco es definir un polinomio maestro $P(x)$ mediante

$$P(x) \equiv \prod_{n=1}^N (x - x_n) \quad (2.8.7)$$

calcular sus coeficientes, y luego obtener los numeradores y denominadores de los P_j específicos mediante división sintética por el único término supernumerario. (Ver §5.3 para más sobre la división sintética.) Dado que cada división es solo un proceso de orden N , el procedimiento total es de orden N^2 .

Debe advertirse que los sistemas de Vandermonde son notoriamente mal condicionados, por su propia naturaleza. (Como una anticipación lateral a §5.8, la razón es la misma que hace que el ajuste de Chebyshev sea tan impresionantemente preciso: existen polinomios de alto orden que se ajustan uniformemente a cero. Por lo tanto, errores de redondeo pueden introducir coeficientes sustanciales en los términos principales de estos polinomios.) Es una buena idea calcular siempre los problemas de Vandermonde en doble precisión.

La rutina para (2.8.2) que sigue es debida a G.B. Rybicki.

Listing 2.1: Rutina para resolver el sistema de Vandermonde (2.8.2)

```

1  #include "nrutil.h"
2
3  void vander(double x[], double w[], double q[], int n)
4  /* Resuelve el sistema lineal de Vandermonde
5   \sum_{i=1}^N x_i^{k-1} w_i = q_k \quad (k = 1, \dots, N).
6   La entrada consiste en los vectores x[1..n] y q[1..n];
7   el vector w[1..n] es la salida. */
8  {
9     int i,j,k;
10    double b,s,t,xx;
11    double *c;
12
13    c=dvector(1,n);
14    if (n == 1) w[1]=q[1];
15    else {
16        for (i=1;i<=n;i++) c[i]=0.0;           // Inicializa el arreglo.
17        c[n] = -x[1];                          // Los coeficientes del polinomio maestro
18                                              // se encuentran por recurrencia.
19
20        for (i=2;i<=n;i++) {
21            xx = -x[i];
22            for (j=(n+1-i);j<=(n-1);j++) c[j] += xx*c[j+1];
23            c[n] += xx;
24        }
25
26        for (i=1;i<=n;i++) {                   // Cada subfactor a su vez
27            xx=x[i];
28            t=b=1.0;
29            s=q[n];
30            for (k=n;k>2;k--) {                 // se divide sintéticamente,
31                b=c[k]+xx*b;                   // se multiplica por el lado derecho,
32                s = q[k-1]*b;                  // y se suministra un denominador.
33                t=xx*t+b;
34            }
35            w[i]=s/t;
36        }
37    }
38    free_dvector(c,1,n);
39 }

```

2.4.2. Matrices de Toeplitz

Una matriz de Toeplitz de tamaño $N \times N$ se especifica dando $2N-1$ números R_k , $k = -N+1, \dots, -1, 0, 1, \dots, N-1$. Esos números se colocan entonces como elementos de matriz constantes a lo largo de las diagonales (de la esquina superior izquierda a la inferior derecha) de la matriz:

$$\begin{bmatrix} R_0 & R_{-1} & R_{-2} & \cdots & R_{-(N-2)} & R_{-(N-1)} \\ R_1 & R_0 & R_{-1} & \cdots & R_{-(N-3)} & R_{-(N-2)} \\ R_2 & R_1 & R_0 & \cdots & R_{-(N-4)} & R_{-(N-3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ R_{N-2} & R_{N-3} & R_{N-4} & \cdots & R_0 & R_{-1} \\ R_{N-1} & R_{N-2} & R_{N-3} & \cdots & R_1 & R_0 \end{bmatrix} \quad (2.8.8)$$

El problema lineal de Toeplitz puede entonces escribirse como

$$\sum_{j=1}^N R_{i-j} x_j = y_i \quad (i = 1, \dots, N) \quad (2.8.9)$$

donde las x_j , $j = 1, \dots, N$, son las incógnitas que deben resolverse.

La matriz de Toeplitz es simétrica si $R_k = R_{-k}$ para todo k . Levinson [?] desarrolló un algoritmo para la solución rápida del problema de Toeplitz simétrico, mediante un *método de ampliación*, es decir,

$$\sum_{j=1}^M R_{i-j} x_j^{(M)} = y_i \quad (i = 1, \dots, M) \quad (2.8.10)$$

una vez para $M = 1, 2, \dots$ hasta que $M = N$, el resultado deseado, finalmente se alcanza. El vector $x^{(M)}$ es el resultado en la etapa M , y se convierte en la respuesta deseada sólo cuando se alcanza N .

El método de Levinson está bien documentado en textos estándar (por ejemplo, [?]). El hecho útil de que el método generaliza al caso no simétrico parece ser menos conocido. A riesgo de exceso de detalle, por lo tanto damos una derivación aquí, debida a G.B. Rybicki.

Al seguir una recurrencia del paso M al paso $M+1$ encontramos que nuestra solución en desarrollo $x^{(M)}$ cambia de esta manera:

$$\sum_{j=1}^M R_{i-j} x_j^{(M)} = y_i \quad i = 1, \dots, M \quad (2.8.11)$$

se convierte en

$$\sum_{j=1}^M R_{i-j} x_j^{(M+1)} + R_{i-(M+1)} x_{M+1}^{(M+1)} = y_i \quad i = 1, \dots, M+1 \quad (2.8.12)$$

Al eliminar y_i encontramos

$$\sum_{j=1}^M R_{i-j} \left(x_j^{(M)} - \frac{x_j^{(M+1)}}{x_{M+1}^{(M+1)}} x_{M+1}^{(M)} \right) = R_{i-(M+1)} \quad i = 1, \dots, M \quad (2.8.13)$$

o dejando que $i \rightarrow M+1-i$ y $j \rightarrow M+1-j$,

$$\sum_{j=1}^M R_{1-j} G_j^{(M)} = R_{-i} \quad (2.8.14)$$

donde

$$G_j^{(M)} = \frac{x_{M+1-j}^{(M)} - x_{M+1-j}^{(M+1)}}{x_{M+1}^{(M+1)}} \quad (2.8.15)$$

Para expresarlo de otra manera,

$$x_j^{(M+1)} = x_j^{(M)} - \frac{x_{M+1-j}^{(M)} - G_j^{(M)}}{x_{M+1}^{(M+1)}} \quad j = 1, \dots, M \quad (2.8.16)$$

Así, si podemos usar recursión para encontrar las cantidades de orden M $x_j^{(M)}$ y $G_j^{(M)}$ y la única cantidad de orden $M+1$ $x_{M+1}^{(M+1)}$, todas las otras cantidades de orden $M+1$ seguirán. Afortunadamente, la cantidad $x_{M+1}^{(M+1)}$ se sigue de la ecuación (2.8.12) con $i = M+1$,

$$\sum_{j=1}^M R_{M+1-j} x_j^{(M)} + R_0 x_{M+1}^{(M+1)} = y_{M+1} \quad (2.8.17)$$

Para las cantidades de orden $M+1$ desconocidas $x_j^{(M+1)}$ podemos sustituir las cantidades de orden anterior en G así

$$G_j^{(M+1)} = \frac{x_{M+1-j}^{(M)} - x_{M+1-j}^{(M+1)}}{x_{M+1}^{(M+1)}} \quad (2.8.18)$$

El resultado de esta operación es

$$x_{M+1}^{(M+1)} = \frac{\sum_{j=1}^M R_{M+1-j} x_j^{(M)} - y_{M+1}}{\sum_{j=1}^M R_{M+1-j} G_j^{(M)} - R_0} \quad (2.8.19)$$

El único problema restante es desarrollar una relación de recurrencia para G . Antes de hacerlo, debemos señalar que en realidad existen dos conjuntos distintos de soluciones para el problema lineal original de una matriz no simétrica, a saber, las soluciones por la derecha (que hemos estado discutiendo) y las soluciones por la izquierda z_i . El formalismo para las soluciones por la izquierda difiere sólo en que tratamos con las ecuaciones

$$\sum_{j=1}^M R_{j-i} z_j^{(M)} = y_i \quad i = 1, \dots, M \quad (2.8.20)$$

Entonces, la misma secuencia de operaciones en este conjunto conduce a

$$\sum_{j=1}^M R_{i-j} H_j^{(M)} = R_i \quad (2.8.21)$$

donde

$$H_j^{(M)} = \frac{z_{M+1-j}^{(M)} - z_{M+1-j}^{(M+1)}}{z_{M+1}^{(M+1)}} \quad (2.8.22)$$

(compárese con 2.8.14 – 2.8.15). La razón para mencionar las soluciones por la izquierda ahora es que, por la ecuación (2.8.21), los H_j satisfacen exactamente la misma ecuación que los x_j excepto por la sustitución $y_i \rightarrow R_i$ en el lado derecho. Por lo tanto podemos deducir rápidamente de la ecuación (2.8.19) que

$$H_{M+1}^{(M+1)} = \frac{\sum_{j=1}^M R_{M+1-j} H_j^{(M)} - R_{M+1}}{\sum_{j=1}^M R_{M+1-j} G_j^{(M)} - R_0} \quad (2.8.23)$$

De la misma forma, G satisface la misma ecuación que z , excepto por la sustitución $y_i \rightarrow -R_{i-1}$. Esto da

$$G_{M+1}^{(M+1)} = \frac{\sum_{j=1}^M R_{j-M-1} G_j^{(M)} - R_{-M-1}}{\sum_{j=1}^M R_{j-M-1} H_j^{(M)} - R_0} \quad (2.8.24)$$

El mismo “morfismo” también convierte las ecuaciones (2.8.16), y su contraparte para z , en las ecuaciones finales

$$G_j^{(M+1)} = G_{M+1-j}^{(M)} - G_{M+1}^{(M+1)} H_{M+1-j}^{(M)} \quad (2.8.25)$$

$$H_j^{(M+1)} = H_j^{(M)} - H_{M+1}^{(M+1)} G_{M+1-j}^{(M)} \quad (2.8.26)$$

Ahora, comenzando con los valores iniciales

$$x_1^{(1)} = y_1/R_0, \quad G_1^{(1)} = R_{-1}/R_0, \quad H_1^{(1)} = R_1/R_0, \quad (2.8.26)$$

podemos continuar recursivamente. En cada etapa M usamos las ecuaciones (2.8.23) y (2.8.24) para encontrar $H_{M+1}^{(M+1)}$, $G_{M+1}^{(M+1)}$, y luego la ecuación (2.8.25) para encontrar los otros componentes de $H^{(M+1)}$, $G^{(M+1)}$.

A partir de ahí los vectores $x^{(M+1)}$ y/o $z^{(M+1)}$ son fácilmente calculados. El programa siguiente hace esto. Incorpora las ecuaciones (2.8.25) en

$$H_{M+1-j}^{(M+1)} = H_{M+1-j}^{(M)} - H_{M+1}^{(M+1)} G_j^{(M)} \quad (2.8.27)$$

de modo que la computación pueda hacerse “en el lugar”.

Nótese que el programa anterior falla si $R_0 = 0$. De hecho, porque el método de ampliación no permite pivotaje, el algoritmo fallará si alguno de los menores principales diagonales de la matriz de Toeplitz original se desvía. (Compárese con la discusión del algoritmo tridiagonal en §2.4.) Si el algoritmo falla, su mezcla puede estar seriamente desbalanceada — usted podría necesitar simplemente pivotar o introducir métodos más generales tales como descomposiciones LU .

La rutina que implementa las ecuaciones (2.8.23)–(2.8.27) también es de Rybicki. Nótese que en la rutina, $r[n+j]$ es igual a R_j como arriba, de modo que los subíndices en el arreglo r varían de 1 a $2N - 1$.

Listing 2.2: Implementación del algoritmo de Toeplitz

```

1 #include "nrutil.h"
2 #define FREERETURN {free_vector(h,1,n);free_vector(g,1,n);return;}
3
4 void toeplz(float r[], float x[], float y[], int n)
5 /* Resuelve el sistema de Toeplitz  $\sum_{j=1}^N R_{(N+i-j)} x_j = y_i$   $(i = 1, \dots, N)$ .
6    La matriz de Toeplitz no necesita ser simétrica. y[1..n] y r[1..2*n-1] son los
7    arreglos de entrada;
8    x[1..n] es el arreglo de salida. */
9 {
10     int j,k,m,m1,m2;
11     float pp,pt1,pt2,qq,qt1,qt2,sd,sgd,sgn,shn,sxn;
12     float *g,*h;
13
14     if (r[n] == 0.0) nrerror("toeplz-1 menor principal singular");
15     g=vector(1,n);
16     h=vector(1,n);
17     x[1]=y[1]/r[n]; /* Inicializar la recurrencia. */
18     if (n == 1) FREERETURN
19     g[1]=r[n-1]/r[n];
20     h[1]=r[n+1]/r[n];
21
22     for (m=1;m<n;m++) { /* Bucle principal de la recurrencia. */
23         m1=m+1;
24         sxn = -y[m1]; /* Calcular numerador y denominador para x, */
25         sd = -r[n];
26         for (j=1;j<=m;j++) {
27             sxn += r[n+m1-j]*x[j];
28             sd += r[n+m1-j]*g[m+1-j];
29         }
30         if (sd == 0.0) nrerror("toeplz-2 menor principal singular");
31         x[m1]=sxn/sd; /* de donde x. */
32         for (j=1;j<=m;j++) x[j] -= x[m1]*g[m+1-j];
33         if (m1 == n) FREERETURN
34         sgn = -r[n-m1]; /* Calcular numerador y denominador para G y H, */
35         shn = -r[n+m1];
36         sgd = -r[n];
37         for (j=1;j<=m;j++) {
38             sgn += r[n+j-m1]*g[j];
39             shn += r[n-m1-j]*h[j];
40             sgd += r[n+j-m1]*h[m+1-j];
41         }
42         if (sgd == 0.0) nrerror("toeplz-3 menor principal singular");
43         g[m1]=sgn/sgd; /* de donde G y H. */
44         h[m1]=shn/sd;
45         k=m;
46         m2=(m+1) >> 1;
47         pp=g[m1];
48         qq=h[m1];
49         for (j=1;j<=m2;j++) {
50             pt1=g[j];
51             pt2=g[k];
52             qt1=h[j];
53             qt2=h[k];
54             g[j]=pt1-pp*qt2;
55             g[k--]=pt2-pp*qt1;
56             h[j]=qt1-qq*pt2;
57             h[k--]=qt2-qq*pt1;
58         }
59     } /* Volver para otra recurrencia. */
60     nrerror("toeplz - no debería llegar aquí!");
61 }

```

Si te dedicas a resolver sistemas de Toeplitz *muy* grandes, deberías informarte sobre los llamados algoritmos “nuevos y rápidos”, que requieren solo del orden de $N(\log N)^2$ operaciones, en comparación con los N^2 del método de Levinson. Estos métodos son demasiado complicados para incluirse aquí.

Capítulo 3

Interpolación y extrapolación

3.1. Introducción

A veces conocemos el valor de una función $f(x)$ en un conjunto de puntos $x_1, x_2, \dots, x_N$ (digamos, con $x_1 < \dots < x_N$), pero no tenemos una expresión analítica para $f(x)$ que nos permita calcular su valor en un punto arbitrario. Por ejemplo, los valores $f(x_i)$ podrían provenir de una medición física o de un cálculo numérico prolongado que no puede expresarse en una fórmula funcional simple. A menudo los x_i están equiespaciados, aunque no necesariamente.

La tarea ahora es estimar $f(x)$ para un x arbitrario, trazando en algún sentido una curva suave a través (y quizá más allá) de los x_i . Si el x deseado se encuentra entre el menor y el mayor de los x_i , el problema se llama *interpolación*; si está fuera de ese rango, se llama *extrapolación*, lo cual es considerablemente más arriesgado (como muchos ex-analistas del mercado de valores pueden atestiguar).

Los esquemas de interpolación y extrapolación deben modelar la función, entre y más allá de los puntos conocidos, mediante alguna forma funcional plausible. Esta forma debe ser lo suficientemente general como para poder aproximar clases amplias de funciones que puedan surgir en la práctica. Con mucho, las más comunes entre las formas funcionales utilizadas son los polinomios (§3.1). Las funciones racionales (cocientes de polinomios) también resultan ser extremadamente útiles (§3.2). Las funciones trigonométricas, senos y cosenos, dan lugar a la *interpolación trigonométrica* y a métodos de Fourier relacionados, que abordaremos en los Capítulos 12 y 13.

Existe una amplia literatura matemática dedicada a teoremas sobre qué tipos de funciones pueden aproximarse bien mediante funciones de interpolación. Estos teoremas son, lamentablemente, casi completamente inútiles en el trabajo cotidiano: si conocemos lo suficiente sobre nuestra función como para aplicar un teorema con poder, por lo general no estamos en el triste estado de tener que interpolar en una tabla de valores.

La interpolación está relacionada con, pero es distinta de, la *aproximación de funciones*. Esta última tarea consiste en encontrar una función aproximada (pero fácilmente computable) para usar en lugar de una más complicada. En el caso de la interpolación, se te dan los valores de la función en puntos que *no has elegido tú mismo*. En el caso de la aproximación de funciones, puedes calcular f en cualquier conjunto de puntos deseado para el propósito de construir tu aproximación. Tratamos la aproximación de funciones en el Capítulo 5.

Es fácil encontrar funciones patológicas que se burlan de cualquier esquema de interpolación. Considera, por ejemplo, la función

$$f(x) = 3x^2 + \frac{1}{\pi^4} \ln [(\pi - x)^2] + 1 \quad (3.0.1)$$

que es bien comportada en todo lugar excepto en $x = \pi$, donde es muy levemente singular, y toma solo valores negativos en ese punto. Cualquier interpolación basada en los valores $x = 3, 13, 3, 14, 3, 15, 3, 16$, seguramente dará una muy mala respuesta para el valor $x = 3, 1416$, ¡aunque una gráfica de esos cinco puntos parezca muy suave! (Pruébalo en tu calculadora.)

Dado que pueden esconderse patologías en cualquier parte, es altamente deseable que toda rutina de interpolación o extrapolación proporcione una estimación de su propio error. Tal estimación nunca puede ser infalible, por supuesto. Podríamos tener una función que, por razones conocidas solo por su creador, se desvía de forma salvaje e inesperada entre dos puntos tabulados. La interpolación siempre presupone cierto grado de suavidad para la función interpolada, pero dentro de ese marco de suposiciones, pueden detectarse desviaciones de la suavidad.

Conceptualmente, el proceso de interpolación tiene dos etapas: (1) Ajustar una función de interpolación a los datos tabulados proporcionados. (2) Evaluar esa función en el punto objetivo x .

Sin embargo, este método de dos etapas generalmente no es el mejor en la práctica. Típicamente es computacionalmente menos eficiente, y más susceptible a errores de redondeo, que los métodos que construyen una estimación funcional $f(x)$ directamente desde los N valores tabulados cada vez que se desea un nuevo valor. La mayoría de los esquemas prácticos comienzan con un valor cercano $f(x_i)$, luego agregan una secuencia de correcciones (esperablemente decrecientes), conforme se incorpora la información de otros valores $f(x_i)$. El procedimiento típicamente toma $\mathcal{O}(N^2)$ operaciones. Si todo se comporta bien, la última corrección será la más pequeña, y puede usarse como una cota informal (aunque no rigurosa) del error.

En el caso de la interpolación polinómica, a veces sucede que los coeficientes del polinomio interpolador son de interés, aunque su uso al *evaluar* la función interpolada debe ser desaconsejado. Abordamos este tema explícitamente en la §3.5.

La interpolación local, usando un número finito de puntos “vecinos más cercanos”, proporciona valores interpolados $f(x)$ que, en general, no tienen derivadas continuas de primer o mayor orden. Esto sucede porque, a medida que x cruza los valores tabulados x_i , el esquema de interpolación cambia de cuáles puntos tabulados son los “más cercanos”. (A menudo, estas interpolaciones locales se implementan como esquemas de “por tramos”). Habrá una discontinuidad en la derivada cada vez que el punto interpolado cruce el dominio de una de estas regiones.

En situaciones donde la continuidad de las derivadas es importante, debe utilizarse la interpolación “más rígida” proporcionada por una función llamada *spline*. Un spline es un polinomio entre cada par de puntos tabulados, pero cuyos coeficientes se determinan de forma “ligeramente” no local. Esta no localidad está diseñada para garantizar suavidad global en la función interpolada hasta cierto orden de derivada. Los splines cúbicos (§3.3) son los más populares. Producen una función interpolada que es continua hasta la segunda derivada. Los splines tienden a ser más estables que los polinomios, con menor posibilidad de oscilaciones salvajes entre los puntos tabulados.

El número de puntos (menos uno) utilizados en un esquema de interpolación se llama el *orden* de la interpolación. Aumentar el orden no necesariamente mejora la precisión, especialmente en la interpolación polinómica. Si los puntos añadidos están alejados del punto de interés x , el polinomio de orden superior resultante, con sus puntos adicionales de restricción, tiende a oscilar ampliamente entre los valores tabulados. Esta oscilación puede no tener relación alguna con el comportamiento de la “verdadera” función (véase Figura 3.0.1). Por supuesto, añadir puntos *cercanos* al punto deseado suele ayudar, pero una malla más fina implica una tabla de valores más extensa, lo cual no siempre está disponible.

A menos que exista evidencia sólida de que la función interpolada es similar en forma a la función verdadera f , es buena idea ser cauteloso con la interpolación de orden alto. Recomendamos con entusiasmo interpolaciones con 3 o 4 puntos, quizá toleramos 5 o 6; pero rara vez vamos más allá de eso, a menos que haya un monitoreo bastante riguroso del error estimado.

Cuando tu tabla de valores contiene muchos más puntos que el orden deseable de interpolación, debes comenzar cada interpolación con una búsqueda de la ubicación “local” correcta en la tabla. Aunque no es estrictamente parte del tema de interpolación, esta tarea es lo suficientemente importante (y suficientemente mal hecha con frecuencia) como para que le dediquemos la §3.4.

Las rutinas dadas para interpolación son también rutinas para extrapolación. Una aplicación importante, en el Capítulo 16, es su uso en la integración de ecuaciones diferenciales ordinarias. Allí, se tiene un cuidado considerable con el monitoreo de errores. De lo contrario, los peligros de la extrapolación no pueden ser exagerados: Una función de interpolación, que por fuerza es también una función de extrapolación, típicamente se volverá loca cuando el argumento x esté fuera del rango de valores tabulados por más de la separación típica entre puntos tabulados.

La interpolación puede hacerse en más de una dimensión, por ejemplo, para una función $f(x, y, z)$. La interpolación multidimensional se logra a menudo mediante una secuencia de interpolaciones unidimensionales. Discutimos esto en §3.6.

3.2. Interpolación y Extrapolación Polinomial

Por dos puntos pasa una única línea. Por tres puntos pasa una única parábola. Etcétera. El polinomio interpolante de grado $N - 1$ que pasa por los N puntos $y_1 = f(x_1)$, $y_2 = f(x_2)$, $\dots$, $y_N = f(x_N)$ está dado explícitamente por la fórmula clásica de Lagrange,

$$P(x) = \frac{(x - x_2)(x - x_3) \dots (x - x_N)}{(x_1 - x_2)(x_1 - x_3) \dots (x_1 - x_N)} y_1 + \frac{(x - x_1)(x - x_3) \dots (x - x_N)}{(x_2 - x_1)(x_2 - x_3) \dots (x_2 - x_N)} y_2 \\ + \dots + \frac{(x - x_1)(x - x_2) \dots (x - x_{N-1})}{(x_N - x_1)(x_N - x_2) \dots (x_N - x_{N-1})} y_N \quad (3.1.1)$$

Hay N términos, cada uno un polinomio de grado $N - 1$ y cada uno construido para ser cero en todos los x_i excepto uno, en el cual está construido para ser igual a y_i .

No está muy mal implementar directamente la fórmula de Lagrange, pero tampoco es lo más correcto. El algoritmo resultante no da una estimación de error, y también es algo incómodo de programar. Un algoritmo mucho mejor (para construir el mismo polinomio interpolante único) es el *algoritmo de Neville*, estrechamente relacionado y a veces confundido con el *algoritmo de Aitken*, este último considerado ahora obsoleto.

Sea P_1 el valor en x del único polinomio de grado cero (es decir, una constante) que pasa por el punto (x_1, y_1) ; así que $P_1 = y_1$. Igualmente definimos $P_2, P_3, \dots, P_N$. Ahora sea P_{12} el valor en x del único polinomio de grado uno que pasa por (x_1, y_1) y (x_2, y_2) . Igualmente $P_{23}, P_{34}, \dots, P_{(N-1)N}$. Similarmente, para polinomios de orden superior, hasta $P_{123\dots N}$, que es el valor del polinomio interpolante único evaluado en x con todos los N puntos, i.e., el valor deseado.

Los diversos P forman una “tabla” con “antecedentes” a la izquierda que conducen a un solo “descendiente” en el extremo derecho. Por ejemplo, con $N = 4$,

$$\begin{array}{rcll} x_1 : & y_1 = P_1 & & \\ x_2 : & y_2 = P_2 & P_{12} & \\ x_3 : & y_3 = P_3 & P_{23} & P_{123} \\ x_4 : & y_4 = P_4 & P_{34} & P_{234} \\ & & P_{1234} & \end{array} \quad (3.1.2)$$

El algoritmo de Neville es una forma recursiva de llenar los números en la tabla una columna a la vez, de izquierda a derecha. Está basado en la relación entre un “hijo” P y sus dos “padres,”

$$P_{i(i+1)\dots(i+m)} = \frac{(x - x_{i+m})P_{i(i+1)\dots(i+m-1)} + (x_i - x)P_{(i+1)(i+2)\dots(i+m)}}{x_i - x_{i+m}} \quad (3.1.3)$$

Esta recurrencia funciona porque los dos padres ya concuerdan en los puntos $x_{i+1} \dots x_{i+m-1}$.

Una mejora a la recurrencia (3.1.3) es llevar un control de las pequeñas *diferencias* entre padres e hijos, a saber, definir (para $m = 1, 2, \dots, N - 1$),

$$C_{m,i} \equiv P_{i\dots(i+m)} - P_{i\dots(i+m-1)} \quad D_{m,i} \equiv P_{i\dots(i+m)} - P_{(i+1)\dots(i+m)} \quad (3.1.4)$$

Entonces, se pueden derivar fácilmente a partir de (3.1.3) las siguientes relaciones:

$$D_{m+1,i} = \frac{(x_{i+m+1} - x)(C_{m,i+1} - D_{m,i})}{x_i - x_{i+m+1}} \quad C_{m+1,i} = \frac{(x_i - x)(C_{m,i+1} - D_{m,i})}{x_i - x_{i+m+1}} \quad (3.1.5)$$

En cada nivel m , los C 's y D 's son las correcciones que hacen que el polinomio sea de orden superior. La respuesta final $P_{1\dots N}$ es igual a la suma de algún y_i más un conjunto de C 's y/o D 's que forman un camino desde la familia hasta el hijo derecho.

Nota: Hay una rutina para interpolación polinomial o extrapolación desde N puntos de entrada. Nota que las matrices de entrada deben comenzar desde el índice unitario. Si utilizas arreglos con índice cero, asegúrate de reajustar (véase §1.2).

```

1 #include <math.h>
2 #include "nrutil.h"
3
4 void polint(float xa[], float ya[], int n, float x, float *y, float *dy)
5 // Dados los arreglos xa[1..n] y ya[1..n], y un valor x,
6 // esta rutina devuelve un valor y, y una estimación de error dy.
7 // Si P(x) es el polinomio de grado N-1 tal que P(xa_i) = ya_i, i = 1,...,n,
8 // entonces el valor retornado y = P(x).
9 {
10     int i, m, ns = 1;
11     float den, dif, dift, ho, hp, w;
12     float *c, *d;
13
14     dif = fabs(x - xa[1]); // Aquí encontramos el índice ns del punto tabulado más
        cercano
15     c = vector(1, n);
16     d = vector(1, n);
17     for (i = 1; i <= n; i++) {
18         dift = fabs(x - xa[i]);
19         if (dift < dif) {
20             ns = i;
21             dif = dift;
22         }
23         c[i] = ya[i]; // inicializamos el "tableau" de los 'cs y 'ds
24         d[i] = ya[i];
25     }
26     *y = ya[ns--]; // Esta es la aproximación inicial a y.
27
28     for (m = 1; m < n; m++) { // Para cada columna del tableau
29         for (i = 1; i <= n - m; i++) { // recorremos los 'cs y 'ds y los actualizamos
30             ho = xa[i] - x;
31             hp = xa[i + m] - x;
32             w = c[i + 1] - d[i];
33             if ((den = ho - hp) == 0.0) nrerror("Error en la rutina polint");
34             // Este error solo puede ocurrir si dos valores de xa son (dentro del
                redondeo) idénticos
35             den = w / den;
36             d[i] = hp * den; // Aquí se actualizan los 'ds y 'cs
37             c[i] = ho * den;
38         }
39         *dy = (2 * ns < (n - m) ? c[ns + 1] : d[ns--]);
40         // Después de completar cada columna, decidimos qué corrección, c o d, añadir
41         // al valor acumulado de y, es decir, qué camino tomar a través del tableau:
42         // hacia arriba o hacia abajo. Lo hacemos de forma que se tome el camino más
43         // "recto" a través del tableau hasta su ápice, actualizando ns para mantener
44         // el seguimiento. Esta ruta mantiene las aproximaciones parciales centradas
45         // (en la medida de lo posible) en el objetivo x. Lo último añadido es la
            indicación de error.
46         *y += *dy;
47     }
48
49     free_vector(d, 1, n);
50     free_vector(c, 1, n);
51 }

```

A menudo querrás llamar a `polint` con los argumentos ficticios `xa` y `ya` reemplazados por arreglos reales *con desplazamientos*. Por ejemplo, la construcción `polint(&xx[14], &yy[14], 4, x, y, dy)` realiza una interpolación de 4 puntos sobre los valores tabulados `xx[15..18]`, `yy[15..18]`. Para más información sobre esto, consulta el final de la sección §3.4.

3.3. Interpolación y Extrapolación con Funciones Racionales

Algunas funciones no se aproximan bien mediante polinomios, pero *sí* se aproximan bien mediante funciones racionales, es decir, cocientes de polinomios. Denotamos por $R_{i(i+1)\dots(i+m)}$ a una función racional que pasa a través de los $m+1$ puntos $(x_i, y_i), \dots, (x_{i+m}, y_{i+m})$. Más explícitamente, supongamos

$$R_{i(i+1)\dots(i+m)}(x) = \frac{P_\mu(x)}{Q_\nu(x)} = \frac{p_0 + p_1x + \dots + p_\mu x^\mu}{q_0 + q_1x + \dots + q_\nu x^\nu} \quad (3.1)$$

Dado que hay $\mu + \nu + 1$ coeficientes p y q desconocidos (q_0 puede ser arbitrario), debemos tener

$$m+1 = \mu + \nu + 1 \quad (3.2)$$

Al especificar una función racional de interpolación, debes proporcionar el orden deseado tanto del numerador como del denominador.

Las funciones racionales a veces son superiores a los polinomios, en sentido aproximado, debido a su habilidad para modelar funciones con polos, es decir, ceros del denominador de la ecuación (3.2.1). Estos polos podrían ocurrir para valores reales de x_i si la función a interpolar tiene polos reales. Más frecuentemente, la función $f(x)$ es finita para todos los x reales, pero tiene una continuación analítica con polos en el plano complejo x -plano. Dichos polos pueden entonces arruinar una aproximación polinomial, incluso una restringida a valores reales de x , así como pueden arruinar la convergencia de una serie infinita de potencias en x . Si dibujas un círculo en el plano complejo alrededor de tus m puntos tabulados, entonces no deberías esperar que la interpolación polinomial sea buena a menos que el polo más cercano esté bastante alejado del círculo. Una aproximación racional, en contraste, se mantendrá “buena” siempre que tenga suficientes potencias de x en su denominador para anular (o cancelar) cualquier polo cercano.

Para el problema de interpolación, una función racional se construye para pasar por un conjunto elegido de valores tabulados. Sin embargo, también deberíamos mencionar que las aproximaciones racionales también pueden usarse en trabajo analítico. Uno a veces construye una aproximación racional con el criterio de que la función racional de la ecuación (3.2.1) misma tenga una expansión en serie de potencias que concuerde con los primeros $m+1$ términos de la expansión en serie de potencias de la función deseada $f(x)$. Esto se llama *aproximación de Padé*, y se discute en la §5.12.

Bulirsch y Stoer encontraron un algoritmo de Neville que realiza interpolación racional sobre una tabla de datos tabulados. Una tabla como la que se usó para Neville se construye columna por columna, conduciendo a un resultado y a una estimación del error. El algoritmo racional básico se llama *interpolación racional de forma diagonal*, porque está basado en una secuencia diagonal (si m es par) o con el grado una unidad mayor (si m es impar), y construida como en la ecuación (3.1.3), con una

$$R_{i(i+1)\dots(i+m)} = R_{i(i+1)\dots(i+(m-1))} + \frac{(x - x_i)}{(x - x_{i+m})} \left(1 - \frac{R_{i(i+1)\dots(i+m)}}{R_{i(i+1)\dots(i+(m-1))}} \right)^{-1} \quad (3.3)$$

Esta recurrencia genera las funciones racionales que pasan por $m+1$ puntos desde aquellas que pasan por m puntos (y el término $R_{i(i+1)\dots(i+m)}$ en la ecuación 3.2.3) $m-1$ puntos. Se inicia con

$$R_i = y_i \quad (3.4)$$

y con

$$R \equiv R_{i(i+1)\dots(i+m)} \quad \text{con } m = -1 \Rightarrow 0 \quad (3.5)$$

Ahora, exactamente como en las ecuaciones (3.1.4) y (3.1.5) anteriores, podemos convertir la recurrencia (3.2.3) en una que involucra sólo las pequeñas diferencias:

$$C_{m,i} \equiv R_{i\dots(i+m)} - R_{i\dots(i+m-1)} \quad (3.6)$$

$$D_{m,i} \equiv R_{i\dots(i+m)} - R_{(i+1)\dots(i+m)} \quad (3.7)$$

Nótese que éstas satisfacen la relación

$$C_{m+1,i} - D_{m+1,i} = C_{m,i+1} - D_{m,i} \quad (3.8)$$

lo cual es útil para demostrar las recurrencias

$$D_{m+1,i} = \left(\frac{C_{m,i+1} - D_{m,i}}{\frac{x - x_i}{x - x_{i+m+1}}} \right) \quad (3.9)$$

$$C_{m+1,i} = \left(\frac{C_{m,i+1} - D_{m,i}}{\frac{x - x_{i+m+1}}{x - x_i}} \right) \quad (3.10)$$

Esta recurrencia se implementa en la siguiente función, cuyo uso es análogo en todo aspecto a `polint` en la §3.1. Nótese nuevamente que se asume que los arreglos de entrada tienen offset de una unidad (§1.2).

```

1 #include <math.h>
2 #include "nrutil.h"
3
4 #define TINY 1.0e-25 // Un número muy pequeño
5 #define FREERETURN {free_vector(d,1,n);free_vector(c,1,n);return;}
6
7 void ratint(float xa[], float ya[], int n, float x, float *y, float *dy)
8 // Dados los arreglos xa[1..n] y ya[1..n], y un valor x, esta rutina
9 // devuelve un valor y y una estimación de precisión dy.
10 // El valor devuelto es el de la función racional diagonal evaluada en x,
11 // la cual pasa por los n puntos (xa_i, ya_i), i = 1..n
12 {
13     int m,i,ns=1;
14     float w,t,hh,h,dd,*c,*d;
15
16     c=vector(1,n);
17     d=vector(1,n);
18     hh=fabs(x-xa[1]);
19     for (i=1;i<=n;i++) {
20         h=fabs(x-xa[i]);
21         if (h == 0.0) {
22             *y=ya[i];
23             *dy=0.0;
24             FREERETURN
25         } else if (h < hh) {
26             ns=i;
27             hh=h;
28         }
29         c[i]=ya[i];
30         d[i]=ya[i]+TINY; // TINY es necesario para evitar una rara división 0/0
31     }
32     *y=ya[ns--];
33     for (m=1;m<n;m++) {
34         for (i=1;i<=n-m;i++) {
35             w=d[i+1]-d[i];
36             h=xa[i+m]-x; // h nunca será cero, pues se verificó en el ciclo anterior
37             t=(xa[i]-x)*w/h;
38             dd=t-c[i+1];
39             if (dd == 0.0) nrerror("Error en la rutina ratint");
40             // Esta condición de error indica que la función interpoladora
41             // tiene un polo en el valor solicitado de x.
42             dd=w/dd;
43             d[i]=c[i+1]*dd;
44             c[i]=t*dd;
45         }
46         *y += (*dy=(2*ns < (n-m) ? c[ns+1] : d[ns--]));
47     }
48     FREERETURN
49 }
```

3.4. Interpolación por Splines Cúbicos

Dada una función tabulada $y_i = y(x_i)$, $i = 1 \dots N$, enfocamos la atención en un intervalo particular, entre x_j y x_{j+1} . La interpolación lineal en ese intervalo da la fórmula de interpolación

$$y = Ay_j + By_{j+1} \quad (3.3.1)$$

donde

$$A = \frac{x_{j+1} - x}{x_{j+1} - x_j} \quad B = 1 - A = \frac{x - x_j}{x_{j+1} - x_j} \quad (3.3.2)$$

Las ecuaciones (3.3.1) y (3.3.2) son un caso especial de la fórmula de interpolación de Lagrange (3.1.1).

Puesto que es (por tramos) lineal, la ecuación (3.3.1) tiene derivada segunda cero en el interior de cada intervalo, y una derivada segunda indefinida o infinita en las abscisas x_j . El objetivo de la interpolación por splines cúbicos es obtener una fórmula de interpolación que sea suave en la primera derivada, y continua en la segunda derivada, tanto dentro de un intervalo como en sus bordes.

Supongamos, contrariamente al hecho, que además de los valores tabulados de y_i , también tenemos valores tabulados para las derivadas segundas de la función, y''_i , es decir, un conjunto de números y''_i . Entonces, dentro de cada intervalo, podemos agregar al lado derecho de la ecuación (3.3.1) un polinomio cúbico cuyas derivadas segundas varíen linealmente desde un valor y''_j en la izquierda hasta un valor y''_{j+1} en la derecha. Al hacerlo, tendremos la continuidad deseada en la derivada segunda. También construiremos el polinomio cúbico para tener los valores cero en x_j y x_{j+1} , y al agregarlo no se arruinará la concordancia con los valores funcionales tabulados y_j y y_{j+1} en los extremos x_j y x_{j+1} .

Un pequeño cálculo auxiliar muestra que solo hay una manera de arreglar esta construcción, reemplazando (3.3.1) por

$$y = Ay_j + By_{j+1} + Cy''_j + Dy''_{j+1} \quad (3.3.3)$$

donde A y B están definidos en (3.3.2) y

$$C = \frac{1}{6}(A^3 - A)(x_{j+1} - x_j)^2 \quad D = \frac{1}{6}(B^3 - B)(x_{j+1} - x_j)^2 \quad (3.3.4)$$

Nótese que la dependencia con respecto a la variable independiente x en las ecuaciones (3.3.3) y (3.3.4) es completamente a través de la dependencia lineal en x de A y B , y (a través de A y B) la dependencia cúbica de x en C y D .

Podemos verificar fácilmente que y'' es, de hecho, la segunda derivada del nuevo polinomio interpolante. Derivamos la ecuación (3.3.3) con respecto a x , usando las definiciones de A, B, C, D para calcular $dA/dx, dB/dx, dC/dx, dD/dx$. El resultado es

$$\frac{dy}{dx} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} + \frac{3A^2 - 1}{6}(x_{j+1} - x_j)y''_j + \frac{3B^2 - 1}{6}(x_{j+1} - x_j)y''_{j+1} \quad (3.3.5)$$

para la primera derivada, y

$$\frac{d^2y}{dx^2} = Ay''_j + By''_{j+1} \quad (3.3.6)$$

para la segunda derivada. Dado que $A = 1$ en x_j , $A = 0$ en x_{j+1} , mientras que B es justo el complemento, la ecuación (3.3.6) muestra que y'' es simplemente la derivada segunda tabulada, y también que la derivada segunda es continua a lo largo de los intervalos (por ejemplo, la frontera entre los dos intervalos x_{j-1}, x_j y x_j, x_{j+1}).

El único problema ahora es que supusimos que los y''_i eran conocidos, cuando en realidad no lo son. Sin embargo, aún no hemos requerido que la primera derivada, calculada a partir de la ecuación (3.3.5), sea continua a través de la frontera entre dos intervalos. La idea clave de un spline cúbico es exigir esta continuidad y usarla para obtener ecuaciones para las segundas derivadas y''_i .

Las ecuaciones requeridas se obtienen igualando la ecuación (3.3.5), evaluada en $x = x_j$ en el intervalo (x_{j-1}, x_j) , a la misma ecuación evaluada en $x = x_j$, pero en el intervalo (x_j, x_{j+1}) . Con algo de reorganización, esto da (para $j = 2, \dots, N-1$):

$$\frac{x_j - x_{j-1}}{6}y''_{j-1} + \frac{x_{j+1} - x_{j-1}}{3}y''_j + \frac{x_{j+1} - x_j}{6}y''_{j+1} = \frac{y_{j+1} - y_j}{x_{j+1} - x_j} - \frac{y_j - y_{j-1}}{x_j - x_{j-1}} \quad (3.3.7)$$

Estas son $N-2$ ecuaciones lineales en los N incógnitas y''_i , $i = 1, \dots, N$. Por lo tanto, hay una familia de soluciones de dos parámetros posibles.

Para una solución única, necesitamos especificar dos condiciones adicionales, típicamente tomadas como condiciones de frontera en x_1 y x_N . Las formas más comunes de hacerlo son:

- Establecer una o ambas de y_1'' y y_N'' iguales a cero, dando lugar al llamado *spline cúbico natural*, que tiene derivada segunda nula en uno o ambos extremos, o
- Fijar y_1'' y/o y_N'' a valores calculados a partir de la ecuación (3.3.5) de manera que la primera derivada de la función interpolante tenga un valor específico en uno o ambos extremos.

Una razón por la cual los splines cúbicos son especialmente prácticos es que el conjunto de ecuaciones (3.3.7), junto con las dos condiciones de frontera opcionales, no solo es lineal, sino también *tridiagonal*. Cada y_j'' está acoplado solo con sus vecinos más cercanos en $j \pm 1$. Por lo tanto, las ecuaciones pueden resolverse en $\mathcal{O}(N)$ operaciones usando el algoritmo tridiagonal (§2.4). Este algoritmo es lo suficientemente rápido como para integrarse directamente en la rutina de interpolación spline. Esto hace que la rutina no solo sea completamente transparente como implementación de (3.3.7), sino también tan eficiente como otras formas de interpolación que utilizan splines cúbicos (§3.4). Los arreglos que usamos se describen más completamente en la §1.2.

```

1 #include "nrutil.h"
2
3 void spline(float x[], float y[], int n, float yp1, float ypn, float y2[])
4 /* Dados los arreglos x[1..n] e y[1..n] que contienen una función tabulada, es decir, y_i
   = f(x_i),
5 con x_1 < x_2 < ... < x_N, y dados los valores yp1 y ypn para la primera derivada de
   la función
6 interpolante en los puntos 1 y n, respectivamente, esta rutina regresa un arreglo y2
   [1..n] que
7 contiene las segundas derivadas de la función interpolante en los puntos tabulados x_i
   .
8 Si yp1 y/o ypn son iguales o mayores a 1 * 10^30, la rutina interpreta que debe usar
   la
9 condición de frontera para una spline natural, con segunda derivada cero en ese
   extremo. */
10 {
11     int i,k;
12     float p,qn,sig,un,*u;
13
14     u=vector(1,n-1);
15     if (yp1 > 0.99e30) {          // La condición de frontera inferior se establece como "
        natural"
16         y2[1]=u[1]=0.0;
17     } else {                      // o bien se especifica una derivada primera
18         y2[1] = -0.5;
19         u[1]=(3.0/(x[2]-x[1]))*((y[2]-y[1])/(x[2]-x[1])-yp1);
20     }
21
22     // Este es el bucle de descomposición del algoritmo tridiagonal.
23     // y2 y u se usan para almacenamiento temporal de los factores descompuestos.
24     for (i=2;i<n;i++) {
25         sig=(x[i]-x[i-1])/(x[i+1]-x[i-1]);
26         p=sig*y2[i-1]+2.0;
27         y2[i]=(sig-1.0)/p;
28         u[i]=(6.0*((y[i+1]-y[i])/(x[i+1]-x[i]) -
29             (y[i]-y[i-1])/(x[i]-x[i-1]))/(x[i+1]-x[i-1]) - sig*u[i-1])/p;
30     }
31
32     if (ypn > 0.99e30) {          // La condición de frontera superior se establece como "
        natural"
33         qn=un=0.0;
34     } else {                      // o bien se especifica una derivada primera
35         qn=0.5;
36         un=(3.0/(x[n]-x[n-1]))*(ypn-(y[n]-y[n-1])/(x[n]-x[n-1]));
37     }
38
39     y2[n]=(un-qn*u[n-1])/(qn*y2[n-1]+1.0);
40
41     // Este es el bucle de sustitución hacia atrás del algoritmo tridiagonal.
42     for (k=n-1;k>=1;k--) {
43         y2[k]=y2[k]*y2[k+1]+u[k];
44     }

```



```

45
46     free_vector(u,1,n-1);
47 }

```

Es importante entender que el programa `spline` se llama sólo *una vez* para procesar toda una función tabulada en los arreglos x_i y y_i . Una vez hecho esto, los valores de la función interpolada para cualquier valor de x se obtienen mediante llamadas (tantas como se deseen) a una rutina separada llamada `splint` (por “interpolación spline”).

```

1 void splint(float xa[], float ya[], float y2a[], int n, float x, float *y)
2 // Dado los arreglos xa[1..n] y ya[1..n], que tabulan una función (con los 'xas en orden)
3 // y dado el arreglo y2a[1..n], que es la salida del programa spline, y dado un valor de
4 // x,
5 // esta rutina retorna el valor interpolado usando un spline cúbico.
6 {
7     void nerror(char error_text[]);
8     int klo,khi,k;
9     float h,b,a;
10
11     klo=1;
12     khi=n;
13     // Encontraremos la posición correcta en la tabla usando bisección.
14     while (khi-klo > 1) {
15         k=(khi+klo) >> 1;
16         if (xa[k] > x) khi=k;
17         else klo=k;
18     }
19     // klo y khi ahora encierran el valor de entrada x.
20     h=xa[khi]-xa[klo];
21     if (h == 0.0) nerror("Entrada xa inválida en la rutina splint");
22     // Los 'xas deben ser distintos.
23     a=(xa[khi]-x)/h;
24     b=(x-xa[klo])/h;
25     // Ahora se evalúa el polinomio del spline cúbico.
26     *y=a*ya[klo]+b*ya[khi]+(((a*a*a-a)*y2a[klo]+(b*b*b-b)*y2a[khi]))*(h*h)/6.0);
27 }

```

3.5. Cómo buscar en una tabla ordenada

Supón que has decidido usar algún esquema de interpolación, como interpolación polinomial de cuarto orden, para calcular una función $f(x)$ a partir de un conjunto tabulado de x_i y f_i . Entonces necesitarás una forma rápida de encontrar tu posición en la tabla de x_i , dado algún valor particular x en el cual deseas hacer la evaluación. Este problema no pertenece propiamente al análisis numérico, pero ocurre tan frecuentemente en la práctica que sería negligente ignorarlo.

Formalmente, el problema es el siguiente: Dado un arreglo de abscisas $xx[j], j = 1, 2, \dots, n$, con elementos ordenados de forma monótonamente creciente o decreciente, y dado un número x , encuentra un entero j tal que x se encuentre entre $xx[j]$ y $xx[j+1]$. Para esta tarea, definimos elementos ficticios en los extremos del arreglo: $xx[0]$ y $xx[n+1]$ igual a más o menos infinito (en el orden que sea consistente con la monotonía de la tabla). Así, j siempre estará entre 0 y n , inclusive. Un valor de 0 indica que x está fuera de rango por debajo, mientras que n indica que está fuera de rango por arriba.

En la mayoría de los casos, una vez que todo está dicho y hecho, es difícil mejorar el método de *bisección*, que encuentra la posición correcta en un tiempo de aproximadamente $\log_2 n$ intentos. Ya usamos bisección en la rutina de evaluación de splines `splint` en la sección anterior, así que podrías revisarla nuevamente. Aislada, una rutina de bisección luce como esta:

```

1 void locate(float xx[], unsigned long n, float x, unsigned long *j)
2 // Dado un arreglo xx[1..n] y un valor x, retorna un valor j tal que x esté entre xx[j] y
   xx[j+1].
3 // xx debe ser monótono, ya sea creciente o decreciente.
4 // j = 0 o j = n se retorna para indicar que x está fuera de rango.
5 {
6     unsigned long ju, jm, jl;
7     int ascd;
8
9     jl = 0;           // Inicializar el límite inferior
10    ju = n + 1;       // e inferior superior
11    ascd = (xx[n] >= xx[1]); // Determinar si la tabla es ascendente
12
13    while (ju - jl > 1) {           // Mientras no se cumpla la condición
14        jm = (ju + jl) >> 1;       // Calcular punto medio
15        if ((x >= xx[jm]) == ascd)
16            jl = jm;               // Actualizar el límite inferior
17        else
18            ju = jm;               // Actualizar el límite superior
19    }
20
21    if (x == xx[1]) *j = 1;
22    else if (x == xx[n]) *j = n - 1;
23    else *j = jl;                 // Retornar el valor j encontrado
24 }
```

Se asume un arreglo `xx` de tipo `float` con índice desde 1. Para usar `locate` con un arreglo que empiece desde 0, recuerda restar 1 al índice calculado y también ajustar la dirección de memoria de `xx`.

3.5.1. Búsqueda con valores correlacionados

A veces estarás en la situación de buscar muchas veces en una tabla grande, y con abscisas casi idénticas en búsquedas consecutivas. Por ejemplo, podrías estar generando una función que se utiliza en el lado derecho de una ecuación diferencial: La mayoría de los integradores de ecuaciones diferenciales, como veremos en el Capítulo 16, llaman a evaluaciones del lado derecho en puntos que saltan hacia adelante y hacia atrás un poco, pero cuya tendencia se mueve lentamente en la dirección de la integración.

En tales casos, es un desperdicio hacer una bisección completa, *ab initio*, en cada llamada. La siguiente rutina, en cambio, comienza con una posición adivinada en la tabla. Primero “caza”, ya sea hacia arriba o hacia abajo, en incrementos de 1, luego 2, luego 4, etc., hasta que el valor deseado esté acotado. En segundo lugar, biseca dentro del intervalo acotado. En el peor de los casos, esta rutina es aproximadamente un factor de 2 más lenta que `locate` (si la fase de caza se expande para incluir toda la tabla). En el mejor de los casos, puede ser un factor de $\log_2 n$ más rápida que `locate`, si el punto deseado usualmente está bastante cerca del valor adivinado. La Figura 3.4.1 compara las dos rutinas.

```

1 void hunt(float xx[], unsigned long n, float x, unsigned long *jlo)
2 /* Dado un arreglo xx[1..n], y un valor x, retorna un valor jlo tal que x está entre xx[
   jlo] y xx[jlo+1].
3 xx[1..n] debe ser monótono, ya sea creciente o decreciente.
4 jlo=0 o jlo=n se retornan para indicar que x está fuera de rango.
5 jlo en la entrada es tomada como la conjetura inicial para jlo en la salida. */
6 {
7     unsigned long jm, jhi, inc;
8     int ascnd;
9
10    ascnd = (xx[n] >= xx[1]); // Orden ascendente o descendente
11
12    if (*jlo <= 0 || *jlo > n) { // Si la conjetura no es útil
13        *jlo = 0;
14        jhi = n + 1;
15    } else {
16        inc = 1;
17        if ((x >= xx[*jlo]) == ascnd) { // Buscar hacia arriba
18            if (*jlo == n) return;
19            jhi = (*jlo) + 1;
20            while ((x >= xx[jhi]) == ascnd) { // Explora hasta acotar
21                *jlo = jhi;
22                inc *= 2;
23                jhi = (*jlo) + inc;
24                if (jhi > n) {
25                    jhi = n + 1;
26                    break;
27                }
28            }
29        } else { // Buscar hacia abajo
30            if (*jlo == 1) {
31                *jlo = 0;
32                return;
33            }
34            jhi = (*jlo)--;
35
36            while ((x < xx[*jlo]) == ascnd) {
37                jhi = (*jlo);
38                inc *= 2;
39                if (inc >= jhi) {
40                    *jlo = 0;
41                    break;
42                } else *jlo = jhi - inc;
43            }
44        }
45    }
46
47    // Búsqueda final por bisección
48    while ((jhi - (*jlo)) != 1) {
49        jm = (jhi + (*jlo)) >> 1;
50        if ((x >= xx[jm]) == ascnd)

```

```

51         *jlo = jm;
52     else
53         jhi = jm;
54     }
55
56     // Correcciones por si x es exactamente igual a un extremo
57     if (x == xx[n]) *jlo = n - 1;
58     if (x == xx[1]) *jlo = 1;
59 }

```

Después de la búsqueda (After the Hunt)

El problema: Las rutinas `locate` y `hunt` retornan un índice j tal que el valor deseado de x se encuentra entre las entradas de la tabla $xx[j]$ y $xx[j+1]$, donde $xx[1..n]$ es la tabla completa. Pero, para obtener un valor interpolado de m puntos utilizando una rutina como `polint` (§3.1) o `ratint` (§3.2), necesitas proporcionar arreglos xx y yy mucho más cortos, de longitud m .

¿Cómo haces esa conexión?

La solución: Calcula

$$k = \text{IMIN}(\text{IMAX}(j - (m - 1)/2, 1), n + 1 - m)$$

(Las macros `IMIN` e `IMAX` devuelven el mínimo y el máximo de dos argumentos enteros; ver §1.2 y el Apéndice B.)

Esta expresión produce el índice del miembro más a la izquierda de un conjunto de m puntos centrado (en la medida de lo posible) entre j y $j + 1$, pero limitado por 1 en el extremo izquierdo y por n en el derecho. Esto te permite llamar a la rutina de interpolación con las direcciones de los arreglos desplazadas por k , por ejemplo:

```
polint(&xx[k-1], &yy[k-1], m, ...)
```

3.5.2. Coeficientes del Polinomio Interpolador

Ocasionalmente podrías querer conocer no el valor del polinomio interpolador que pasa por un (pequeño) número de puntos, sino los coeficientes de ese polinomio. Un conjunto útil de coeficientes podría ser, por ejemplo, para computar valores simultáneamente interpolados de la función y de varias de sus derivadas (véase §3.7), o para convolver un segmento de la función tabulada con alguna otra función, donde los momentos de esa otra función (es decir, su convolución con potencias de x) se conocen analíticamente.

Sin embargo, asegúrate de que los coeficientes que buscas sean realmente los necesarios. En general, los coeficientes del polinomio interpolador pueden determinarse con mucha menor precisión que su valor en un punto deseado. Por tanto, no es buena idea determinar los coeficientes solo para luego usarlos al calcular valores interpolados. Los valores así calculados podrían no pasar exactamente por los puntos tabulados, mientras que los valores computados por las rutinas de §3.1–§3.3 sí pasan exactamente por tales puntos.

Tampoco debes confundir el polinomio interpolador (y sus coeficientes) con su primo cercano, el mejor ajuste de un polinomio a los datos. El ajuste es un proceso de suavizado, ya que el número de coeficientes ajustados suele ser mucho menor que el número de puntos de datos. Por tanto, los coeficientes ajustados pueden determinarse con precisión incluso en presencia de errores estadísticos en los valores tabulados (véase §14.8). La interpolación, donde el número de coeficientes y el número de puntos tabulados son iguales, toma los valores tabulados como perfectos. Si en realidad contienen errores estadísticos, estos pueden amplificarse en oscilaciones del polinomio interpolador entre los puntos tabulados.

Como antes, tomamos los valores tabulados como $y_i \equiv y(x_i)$. Si el polinomio interpolador se escribe como:

$$y = c_0 + c_1x + c_2x^2 + \cdots + c_Nx^N \quad (3.5.1)$$

entonces los coeficientes c_i deben satisfacer la ecuación lineal:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^N \\ 1 & x_1 & x_1^2 & \cdots & x_1^N \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^N \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_N \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{bmatrix} \quad (3.5.2)$$

Esta es una matriz de Vandermonde, como se describe en §2.8. En principio, se puede resolver la ecuación (3.5.2) mediante técnicas estándar de resolución de ecuaciones lineales (véase §2.3); sin embargo, el método especial derivado en §2.8 es mucho más eficiente para un gran factor de N , por lo que es mucho mejor.

Recuerda que los sistemas con matrices de Vandermonde pueden estar muy mal condicionados. En tales casos, ningún método numérico puede ofrecer una respuesta precisa. Tales dificultades no implican dificultad para encontrar valores interpolados por los métodos de §3.1, pero sí dificultan encontrar *coeficientes*.

Como se indica en §2.8, el índice comienza en 0 debido a G. B. Rybicki. Observa que los arreglos en este capítulo están indexados desde cero.

3.5.3. Otro Método

Otra técnica consiste en hacer uso de la rutina de interpolación de valores de función ya dada (polint §3.1). Si interpolamos (o extrapolamos) para encontrar el valor del polinomio interpolador en $x = 0$, entonces este valor será evidentemente c_0 . Ahora podemos restar c_0 de los valores y_i y dividir cada uno por su correspondiente x_i . Descartando un punto (el que tenga el menor x_i es una buena opción), podemos repetir el procedimiento para encontrar c_1 , y así sucesivamente.

No es evidente de inmediato que este procedimiento sea estable, pero en general hemos encontrado que es *algo más* estable que la rutina anterior. Este método es de orden N^3 , mientras que el anterior era de orden N^2 . Sin embargo, encontrarás que ninguno de los dos funciona muy bien para N grandes, debido a la condición intrínsecamente mal condicionada del problema de Vandermonde. En precisión simple, N hasta 8 o 10 es satisfactorio; aproximadamente el doble en precisión doble.

```

1 #include <math.h>
2 #include "nrutil.h"
3
4 void polcof(float xa[], float ya[], int n, float cof[])
5 /* Dados los arreglos xa[0..n] y ya[0..n] que contienen una función tabulada
6  ya_i = f(xa_i), esta rutina devuelve un arreglo de coeficientes cof[0..n]
7  tal que ya_i = \sum_j cof_j x_a^j */
8 {
9     void polint(float xa[], float ya[], int n, float x, float *y, float *dy);
10    int k,j,i;
11    float xmin,dy,*x,*y;
12
13    x = vector(0,n);
14    y = vector(0,n);
15    for (j = 0; j <= n; j++) {
16        x[j] = xa[j];
17        y[j] = ya[j];
18    }
19
20    for (j = 0; j <= n; j++) {
21        polint(x-1, y-1, n+1-j, 0.0, &cof[j], &dy); // Extrapolamos a x = 0
22        xmin = 1.0e38;
23        k = -1;
24        for (i = 0; i <= n-j; i++) {
25            if (fabs(x[i]) < xmin) {
26                xmin = fabs(x[i]);
27                k = i;
28            }
29            if (x[i] != 0.0) {
30                y[i] = (y[i] - cof[j]) / x[i]; // reducimos todos los términos
31            }
32        }
33        for (i = k+1; i <= n-j; i++) {
34            y[i-1] = y[i]; // eliminamos el valor mínimo
35            x[i-1] = x[i];
36        }
37    }
38
39    free_vector(y,0,n);
40    free_vector(x,0,n);
41 }

```

Si el punto $x = 0$ no se encuentra dentro (o al menos cerca) del rango de los valores tabulados x_i , entonces los coeficientes del polinomio interpolante, en general, pueden volverse muy grandes. Sin embargo, el verdadero “contenido de información” de los coeficientes reside en las pequeñas diferencias que se producen por los grandes valores inducidos por la “traslación”. Esta es una causa de mala condición numérica, lo que resulta en pérdida de cifras significativas y coeficientes mal determinados. Deberías considerar redefinir el origen del problema, para poner $x = 0$ en un lugar sensato.

Otra patología es que, si se intenta una interpolación de grado demasiado alto sobre una función suave, el polinomio interpolante intentará utilizar sus coeficientes de grado alto en combinación con coeficientes grandes y casi perfectamente cancelados entre sí, para ajustarse a los valores tabulados con la mayor precisión posible. Este efecto es el mismo que la tendencia intrínseca del polinomio interpolante a oscilar (violentamente) entre sus puntos de restricción, y estaría presente incluso si la precisión en punto flotante de la máquina fuera infinitamente buena. Las rutinas anteriores `polcoe` y `polcof` tienen sensibilidades ligeramente diferentes a estas patologías que pueden ocurrir.

¿Sigues completamente seguro de que usar los *coeficientes* es una buena idea?

3.5.4. Interpolación en dos o más dimensiones

En la interpolación multidimensional, buscamos una estimación de $y(x_1, x_2, \dots, x_n)$ a partir de una grilla n -dimensional de valores tabulados y y n vectores unidimensionales que dan los valores tabulados de cada una de las variables independientes $x_1, x_2, \dots, x_n$. No consideraremos aquí el problema de interpolar sobre una malla que no es cartesiana, es decir, que tiene valores tabulados en puntos “aleatorios” en el espacio n -dimensional en lugar de en los vértices de una grilla rectangular. Para mayor claridad, consideraremos explícitamente solo el caso de dos dimensiones, siendo los casos de tres o más dimensiones análogos en todos los sentidos.

En dos dimensiones, imaginamos que se nos da una matriz de valores funcionales `ya[1..m][1..n]`. También se nos dan un arreglo `x1a[1..m]`, y un arreglo `x2a[1..n]`. La relación de estas entradas con una función subyacente $y(x_1, x_2)$ es

$$ya[j][k] = y(x1a[j], x2a[k]) \quad (3.11)$$

Queremos estimar, por interpolación, el valor de la función y en algún punto no tabulado (x_1, x_2) .

Un concepto importante es el del *cuadro de grilla* en el que cae el punto (x_1, x_2) , es decir, los cuatro puntos tabulados que rodean el punto interior deseado. Para comodidad, numeraremos estos puntos del 1 al 4, en sentido antihorario comenzando desde la esquina inferior izquierda (véase la Figura 3.6.1). Más precisamente, si

$$x1a[j] \leq x_1 \leq x1a[j+1] \quad x2a[k] \leq x_2 \leq x2a[k+1] \quad (3.12)$$

definen j y k , entonces

$$y_1 \equiv ya[j][k] \quad (3.13)$$

$$y_2 \equiv ya[j+1][k] \quad (3.14)$$

$$y_3 \equiv ya[j+1][k+1] \quad (3.15)$$

$$y_4 \equiv ya[j][k+1] \quad (3.16)$$

La interpolación más simple en dos dimensiones es la *interpolación bilineal* en el cuadro de grilla. Sus fórmulas son:

$$t = (x_1 - x1a[j]) / (x1a[j+1] - x1a[j]) \quad (3.17)$$

$$u = (x_2 - x2a[k]) / (x2a[k+1] - x2a[k]) \quad (3.18)$$

(de modo que t y u estén entre 0 y 1), y

$$y(x_1, x_2) = (1-t)(1-u)y_1 + t(1-u)y_2 + tuy_3 + (1-t)uy_4 \quad (3.19)$$

La interpolación bilineal suele ser “suficientemente buena para fines prácticos gubernamentales”. A medida que el punto de interpolación se desplaza de un cuadro de grilla a otro, el valor de la función interpolada cambia continuamente. Sin embargo, el gradiente de la función interpolada cambia de forma discontinua en los bordes de cada cuadro de grilla.

Existen dos direcciones claramente diferentes que se pueden tomar para ir más allá de la interpolación bilineal hacia métodos de orden superior: se puede usar un orden superior para obtener mayor precisión en la función interpolada (¡para funciones suficientemente suaves!), sin intentar necesariamente corregir la continuidad del

gradiente y las derivadas superiores. O bien, se puede hacer uso de un orden superior para imponer suavidad en algunas de estas derivadas al cruzar los bordes de los cuadros de grilla. A continuación consideraremos cada una de estas dos direcciones por separado.

3.5.5. Orden superior para mayor precisión

La idea básica consiste en descomponer el problema en una sucesión de interpolaciones unidimensionales. Si queremos hacer una interpolación de orden $m - 1$ en la dirección x_1 , y de orden $n - 1$ en la dirección x_2 , primero localizamos un sub-bloque $m \times n$ de la matriz de la función tabulada que contenga nuestro punto deseado (x_1, x_2) . Luego hacemos m interpolaciones unidimensionales en la dirección x_2 , es decir, sobre las filas del sub-bloque, para obtener valores de función en los puntos $(x1a[j], x_2)$, $j = 1, \dots, m$. Finalmente, hacemos una última interpolación en la dirección x_1 para obtener el resultado. Si usamos la rutina de interpolación polinomial `polint` de la §3.1, y un sub-bloque que se presume ya ha sido localizado (y referenciado mediante el apuntador `float **ya`, véase §1.2), el procedimiento se ve así:

```

1 #include "nrutil.h"
2
3 void polin2(float x1a[], float x2a[], float **ya, int m, int n,
4           float x1, float x2, float *y, float *dy)
5 {
6     void polint(float xa[], float ya[], int n, float x, float *y, float *dy);
7     int j;
8     float *ymtmp;
9
10    ymtmp=vector(1,m);
11    for (j=1;j<=m;j++) {
12        polint(x2a,ya[j],n,x2,&ymtmp[j],dy);
13    }
14    polint(x1a,ymtmp,m,x1,y,dy);
15    free_vector(ymtmp,1,m);
16 }
```

3.5.6. Orden Superior para Suavidad: Interpolación Bicúbica

Presentaremos dos métodos que están en uso común, y que no son independientes entre sí. El primero se llama usualmente *interpolación bicúbica*.

La interpolación bicúbica requiere que el usuario especifique en cada punto de la cuadrícula no solo la función $y(x_1, x_2)$, sino también los gradientes $\partial y / \partial x_1 \equiv y_{,1}$, $\partial y / \partial x_2 \equiv y_{,2}$ y la derivada cruzada $\partial^2 y / \partial x_1 \partial x_2 \equiv y_{,12}$. Entonces se puede encontrar una función interpolante que sea *cúbica* en las coordenadas escaladas t y u (ver ecuación 3.6.4), con las siguientes propiedades:

(i) Los valores de la función y de las derivadas especificadas se reproducen exactamente en los puntos de la cuadrícula, y (ii) Los valores de la función y de las derivadas especificadas cambian de manera continua conforme el punto de interpolación cruza de un cuadrado de la cuadrícula a otro.

Es importante entender que nada en las ecuaciones de la interpolación bicúbica requiere que se especifiquen las derivadas adicionales *correctamente*. Las propiedades de suavidad están tautológicamente “forzadas”, y no tienen nada que ver con la “precisión” de las derivadas especificadas. Es un problema separado decidir cómo obtener los valores que se especificarán. Cuanto mejor se haga esto, más *precisa* será la interpolación. Pero será *suave* sin importar cómo se haga.

Lo mejor de todo es conocer las derivadas analíticamente, o poder computarlas por medios numéricos, en los puntos de la cuadrícula. Lo siguiente mejor es determinarlas por diferenciación numérica a partir de los valores funcionales tabulados disponibles en la cuadrícula. El código relevante podría lucir así (utilizando diferencias centradas):

```

1 y1a[j][k] = (ya[j+1][k] - ya[j-1][k]) / (x1a[j+1] - x1a[j-1]);
2 y2a[j][k] = (ya[j][k+1] - ya[j][k-1]) / (x2a[k+1] - x2a[k-1]);
3 y12a[j][k] = (ya[j+1][k+1] - ya[j+1][k-1] - ya[j-1][k+1] + ya[j-1][k-1])
4             / ((x1a[j+1] - x1a[j-1]) * (x2a[k+1] - x2a[k-1]));
```

Para hacer una interpolación bicúbica dentro de un cuadrado de cuadrícula, dados la función y y las derivadas $y_{,1}$, $y_{,2}$, $y_{,12}$ en cada una de las cuatro esquinas del cuadrado, hay dos pasos:

Primero se obtienen dieciséis cantidades c_{ij} , $i, j = 1, \dots, 4$ usando la rutina `bcuint`. (Las fórmulas que obtienen los c a partir de los valores de la función y derivadas son complicadas pero lineales, con coeficientes que, una vez determinados para este método de historial numérico, pueden tabularse y olvidarse).

$$y(x_1, x_2) = \sum_{i=1}^4 \sum_{j=1}^4 c_{ij} t^{i-1} u^{j-1} \quad (3.20)$$

$$y_{,1}(x_1, x_2) = \sum_{i=1}^4 \sum_{j=1}^4 (i-1) c_{ij} t^{i-2} u^{j-1} \left(\frac{dt}{dx_1} \right) \quad (3.21)$$

$$y_{,2}(x_1, x_2) = \sum_{i=1}^4 \sum_{j=1}^4 (j-1) c_{ij} t^{i-1} u^{j-2} \left(\frac{du}{dx_2} \right) \quad (3.22)$$

$$y_{,12}(x_1, x_2) = \sum_{i=1}^4 \sum_{j=1}^4 (i-1)(j-1) c_{ij} t^{i-2} u^{j-2} \left(\frac{dt}{dx_1} \right) \left(\frac{du}{dx_2} \right) \quad (3.23)$$

donde t y u son nuevamente dados por la ecuación (3.6.4).

```

1 void bcucof(float y[], float y1[], float y2[], float y12[],
2           float d1, float d2, float **c)
3 {
4     /* Dados los arreglos y[1..4], y1[1..4], y2[1..4], y12[1..4],
5      que contienen la función, los gradientes y la derivada cruzada
6      en los cuatro puntos de la cuadrícula de una celda rectangular
7      (numerados en sentido antihorario desde la esquina inferior izquierda),
8      y dados d1 y d2, las longitudes de la celda de la cuadrícula en las
9      direcciones 1 y 2, esta rutina retorna la tabla c[1..4][1..4]
10     que es usada por la rutina bcuint para la interpolación bicúbica.
11     */
12
13     static int wt[16][16] = {
14         { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
15         { 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0 },
16         { -3, 0, 0, 3, 0, 0, 0, 0, -2, 0, 0, 2, -1, 0, 0, 1 },
17         { 2, 0, 0, -2, 0, 0, 0, 0, 1, 0, 0, -1, 0, 0, 0, 0 },
18         { 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
19         { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
20         { 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, 0, -3, 0, 0, 0, 0 },
21         { 0, 0, 0, 0, 0, 0, 0, 0, -2, 0, 0, 2, 0, 0, 0, 0 },
22         { 0, 0, 0, 0, -3, 3, 0, 0, -3, 3, 0, 0, -2, 2, 0, 0 },
23         { 9, -9, -9, 9, 6, -6, 6, -6, 3, -3, -3, 3, 2, -2, 2, -2 },
24         { -6, 6, 6, -6, -4, 4, -2, 2, -3, 3, 3, -3, -2, 2, -1, 1 },
25         { 2, -2, 0, 0, 1, -1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
26         { 0, 0, 0, 0, 0, 0, 0, 0, 2, -2, 0, 0, 1, -1, 0, 0 },
27         { -6, 6, 6, -6, -3, 3, 3, -3, -4, 4, 4, -4, -2, 2, 2, -2 },
28         { -4, 4, 4, -4, 2, -2, -2, 2, -2, 2, 2, -2, 1, -1, -1, 1 },
29         { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 }
30     };
31
32     int l, k, j, i;
33     float xx, d1d2, cl[16], x[16];
34
35     d1d2 = d1 * d2;
36
37     // Empaquetar el vector temporal x
38     for (i = 1; i <= 4; i++) {
39         x[i - 1] = y[i];
40         x[i + 3] = y1[i] * d1;
41         x[i + 7] = y2[i] * d2;
42         x[i + 11] = y12[i] * d1d2;
43     }
44
45     // Multiplicación matricial por la tabla almacenada
46     for (i = 0; i <= 15; i++) {
47         xx = 0.0;
48         for (k = 0; k <= 15; k++) xx += wt[i][k] * x[k];
49         cl[i] = xx;
50     }
51 }

```



```

51
52 // Desempaquetar el resultado en la tabla de salida
53 l = 0;
54 for (i = 1; i <= 4; i++) {
55     for (j = 1; j <= 4; j++) {
56         c[i][j] = cl[l++];
57     }
58 }
59 }

```

La implementación de la ecuación (3.6.6), que realiza una interpolación bicúbica, devuelve el valor de la función interpolada y los dos valores de derivadas parciales, y usa la rutina anterior `bcucof`, es simplemente:

```

1 #include "nrutil.h"
2
3 void bcuint(float y[], float y1[], float y2[], float y12[],
4             float x1l, float x1u, float x2l, float x2u,
5             float x1, float x2, float *ansy,
6             float *ansy1, float *ansy2)
7
8 /* Interpolación bicúbica dentro de un cuadrado de malla.
9  Las cantidades de entrada son y, y1, y2, y12 (como se describió en bcucof);
10 x1l y x1u son las coordenadas inferior y superior del cuadrado de la malla
11 en la dirección 1; x2l y x2u en la dirección 2;
12 y x1, x2 son las coordenadas del punto deseado para la interpolación.
13 El valor interpolado de la función se retorna como ansy,
14 y los valores interpolados de las derivadas como ansy1 y ansy2.
15 Esta rutina llama a bcucof.
16 */
17
18 {
19     void bcucof(float y[], float y1[], float y2[], float y12[],
20                 float d1, float d2, float **c);
21     int i;
22     float t, u, d1, d2, **c;
23
24     c = matrix(1,4,1,4);
25     d1 = x1u - x1l;
26     d2 = x2u - x2l;
27     bcucof(y, y1, y2, y12, d1, d2, c); /* Obtener los 'cs */
28
29     if (x1u == x1l || x2u == x2l)
30         nrerror("Bad input in routine bcuint");
31
32     t = (x1 - x1l) / d1; /* Ecuación (3.6.4) */
33     u = (x2 - x2l) / d2;
34
35     *ansy = (*ansy2) = (*ansy1) = 0.0;
36
37     for (i = 4; i >= 1; i--) { /* Ecuación (3.6.6) */
38         *ansy += (*ansy +=
39                 ((c[i][4] * u + c[i][3]) * u + c[i][2]) * u + c[i][1]);
40         *ansy2 += ((*ansy2 += (3.0 * c[i][4] * u + 2.0 * c[i][3]) * u + c[i][2]));
41         *ansy1 += (*ansy1 +=
42                 (3.0 * c[4][i] * t + 2.0 * c[3][i]) * t + c[2][i]);
43     }
44
45     *ansy1 /= d1;
46     *ansy2 /= d2;
47     free_matrix(c, 1, 4, 1, 4);
48 }

```

3.5.7. Mayor orden para suavidad: Spline Bicúbico

La otra técnica común para obtener suavidad en la interpolación bidimensional es el *spline bicúbico*. En realidad, esto es equivalente a un caso especial de interpolación bicúbica: la función interpolante tiene la misma forma funcional que la ecuación (3.6.6); los valores de las derivadas en los puntos de la cuadrícula son, sin embargo, determinados “globalmente” mediante splines unidimensionales. Sin embargo, los splines bicúbicos generalmente se implementan de una forma que luce bastante diferente a las rutinas de interpolación bicúbica anteriores, y más parecida a la rutina `polin2` vista anteriormente: para interpolar un solo valor funcional, se realizan m interpolaciones unidimensionales a lo largo de las filas de la tabla, seguidas de una interpolación unidimensional adicional a lo largo de la nueva columna creada. Es cuestión de gusto (y de balance entre tiempo y memoria) cuánto de este proceso se quiere precomputar y almacenar. En lugar de precomputar y almacenar toda la información de derivadas (como en la interpolación bicúbica), los usuarios de splines típicamente precomputan y almacenan sólo una tabla auxiliar, de segundas derivadas en una sola dirección. Luego solo se necesitan realizar *evaluaciones* de spline (no construcciones) para las m filas de los splines; aún se debe hacer una construcción y una evaluación para la spline de la columna final. (Recuerda que una construcción de spline es un proceso de orden N , mientras que una evaluación de spline es sólo de orden $\log N$ —y eso solo para encontrar el lugar en la tabla!)

Aquí hay una rutina para precomputar la tabla auxiliar de segundas derivadas:

```

1 void splie2(float x1a[], float x2a[], float **ya, int m, int n, float **y2a)
2 /* Dado un array m por n de una función tabulada y_{i,j} = ya[i][j],
3 y valores tabulados independientes x1a[1..m], x2a[1..n], esta rutina construye
4 splines cúbicos naturales unidimensionales de las filas de ya y retorna las
5 segundas derivadas en el array y2a[1..m][1..n]. (El array x1a[1..m] se incluye
6 en el argumento para consistencia con la rutina splin2.) */
7 {
8     void spline(float x[], float y[], int n, float yp1, float ypn, float y2[]);
9     int j;
10
11     for (j=1; j<=m; j++)
12         spline(x2a, ya[j], n, 1.0e30, 1.0e30, y2a[j]); // Valores 1×10^{30} señalan spline
13                                     natural
14 }

```

(Para interpolar sobre un sub-bloque de una matriz más grande, ver §1.2.)

Una vez que la rutina anterior ha sido ejecutada una vez, cualquier número de interpolaciones cúbicas bicúbicas puede ser realizadas por llamadas sucesivas a la siguiente rutina:

```

1 #include "nrutil.h"
2
3 void splin2(float x1a[], float x2a[], float **ya, float **y2a, int m, int n,
4             float x1, float x2, float *y)
5 /* Dados x1a, x2a, ya, y m, n como descrito en splie2 y y2a como producido por
6 esa rutina; y dado un punto de interpolación deseado x1,x2; esta rutina retorna
7 el valor de la función interpolada y mediante interpolación cúbica bicúbica. */
8 {
9     void spline(float xa[], float ya[], int n, float yp1, float ypn, float y2[]);
10    void splint(float xa[], float ya[], float y2a[], int n, float x, float *y);
11    int j;
12    float *ytmp, *ytmp2;
13
14    ytmp=vector(1,m);
15    ytmp2=vector(1,m); // Realiza m evaluaciones de las splines de fila construidas por
16                        splie2
17    for (j=1; j<=m; j++) {
18        splint(x2a, ya[j], y2a[j], n, x2, &ytmp[j]); // splint
19    }
20    spline(x1a, ytmp, m, 1.0e30, 1.0e30, ytmp2); // Construye la spline unidimensional de
21                        columna y la evalúa
22    splint(x1a, ytmp, ytmp2, m, x1, y);
23
24    free_vector(ytmp, 1, m);
25    free_vector(ytmp2, 1, m);
26 }

```

Capítulo 4

Modelado de Datos

4.1. Introducción

Dado un conjunto de observaciones, a menudo se desea condensar y resumir los datos ajustándolos a un “modelo” que depende de parámetros ajustables. A veces, el modelo es simplemente una clase conveniente de funciones, como polinomios o gaussianas, y el ajuste proporciona los coeficientes apropiados. Otras veces, los parámetros del modelo provienen de alguna teoría subyacente que se supone deben satisfacer los datos; ejemplos son coeficientes de ecuaciones de velocidad en una red compleja de reacciones químicas, o elementos orbitales de una estrella binaria. El modelado también puede usarse como una especie de interpolación restringida, donde se quiere extender unos pocos puntos de datos a una función continua, pero con alguna idea subyacente de cómo debería verse esa función.

El enfoque básico en todos los casos es generalmente el mismo: se elige o diseña una *función de méritos* (“función de mérito”, para abreviar) que mide la concordancia entre los datos y el modelo con una elección particular de parámetros. La función de mérito se organiza convencionalmente de manera que valores pequeños representen un buen ajuste. Los parámetros del modelo se ajustan luego para alcanzar un mínimo en la función de mérito, produciendo los *parámetros de mejor ajuste*. Este proceso de ajuste es así un problema de minimización en muchas dimensiones. Esta optimización fue el tema del Capítulo 10; sin embargo, existen métodos especiales, más eficientes, que son específicos del modelado, y discutiremos estos en este capítulo.

Hay cuestiones importantes que van más allá de simplemente encontrar los parámetros de mejor ajuste. Los datos generalmente no son exactos. Están sujetos a *errores de medición* (llamados *ruido* en el contexto del procesamiento de señales). Así, los datos típicos rara vez se ajustan exactamente al modelo que se está utilizando, incluso cuando ese modelo es correcto. Necesitamos medios para evaluar si el modelo es apropiado o no; es decir, necesitamos probar el *grado de ajuste* en relación con algún estándar estadístico útil.

También necesitamos saber con qué precisión se determinan los parámetros por el conjunto de datos. En otras palabras, necesitamos conocer los errores probables de los parámetros de mejor ajuste.

Finalmente, no es raro en el ajuste de datos descubrir que la función de mérito no es unimodal, con un solo mínimo. En algunos casos, podemos estar interesados en el mínimo global en lugar de en cuestiones locales. No se trata de “¿qué tan bueno es este ajuste?”, sino más bien, “¿estoy seguro de que no hay un ajuste *mucho mejor* en alguna otra parte del espacio de parámetros?” Como hemos visto en el Capítulo 10, especialmente en el §10.9, este tipo de problema es generalmente bastante difícil de resolver.

El punto importante que queremos dejar claro es que ajustar parámetros no es el fin de todo el proceso de ajuste de datos. Más bien, la salida del ajuste debe proporcionar (i) parámetros, (ii) estimaciones de error en los parámetros, y (iii) una medida estadística del grado de ajuste. Cuando el tercer elemento sugiere que el modelo es una coincidencia poco probable con los datos, entonces los elementos (i) y (ii) son probablemente inútiles. Desafortunadamente, muchos practicantes de estimación de parámetros nunca pasan del elemento (i). Consideran aceptable un ajuste si una gráfica de los datos y el modelo “se ve bien”. Este enfoque se conoce como *chi-a-ojómetro*. Afortunadamente, sus practicantes reciben lo que merecen.

4.2. Mínimos Cuadrados como un Estimador de Máxima Verosimilitud

Supongamos que estamos ajustando N puntos de datos (x_i, y_i) , $i = 1, \dots, N$, a un modelo que tiene M parámetros ajustables a_j , $j = 1, \dots, M$. El modelo predice una relación funcional entre las variables independientes y dependientes medidas,

$$y(x) = y(x; a_1 \dots a_M) \quad (15.1.1)$$

donde la dependencia con respecto a los parámetros se indica explícitamente en el lado derecho.

Entonces, ¿exactamente qué queremos minimizar para obtener valores ajustados para los a_j ? Lo primero que viene a la mente es el conocido ajuste por mínimos cuadrados,

$$\min_{a_1 \dots a_M} \sum_{i=1}^N [y_i - y(x_i; a_1 \dots a_M)]^2 \quad (15.1.2)$$

Pero, ¿de dónde proviene esto? ¿En qué principios generales se basa? La respuesta a estas preguntas nos lleva al tema de los *estimadores de máxima verosimilitud*.

Dado un conjunto de datos particular de x_i 's y y_i 's, tenemos la sensación intuitiva de que ciertos conjuntos de parámetros $a_1 \dots a_M$ son muy poco probables —aquellos para los cuales la función del modelo $y(x)$ no se parece en nada a los datos— mientras que otros pueden ser muy probables— aquellos que se asemejan mucho a los datos. ¿Cómo podemos cuantificar esta sensación intuitiva? ¿Cómo podemos seleccionar los parámetros que son los “más probables” de ser correctos? No tiene mucho sentido preguntar: “¿Cuál es la probabilidad de que un conjunto particular de parámetros ajustados $a_1 \dots a_M$ sea correcto?” La razón es que no hay un universo estadístico de modelos del cual provienen los parámetros. Solo hay un modelo, el correcto, y un estándar estadístico de uso para evaluar qué tan bien se ajusta.

Dicho esto, podemos reformular la pregunta: “*Dado un conjunto particular de parámetros, ¿cuál es la probabilidad de que este conjunto de datos haya ocurrido?*” Si los y_i toman valores continuos, la probabilidad siempre será cero a menos que se agregue la frase “...más o menos un pequeño Δy en cada punto de datos”. Así que siempre tomamos esta frase como implícita. Si la probabilidad de obtener los datos es infinitesimalmente pequeña, podemos concluir que los parámetros bajo consideración son “improbables” de ser correctos. Por el contrario, nuestra intuición nos dice que los datos no deben ser demasiado improbables para el conjunto correcto de parámetros.

En otras palabras, identificamos la probabilidad de los datos dado los parámetros (lo cual es una cantidad computacionalmente significativa), como la *verosimilitud* de los parámetros dado los datos. Esta identificación es enteramente basada en la intuición. No tiene una base matemática formal en sí misma; como ya mencionamos, la estadística *no* es una rama de las matemáticas.

Una vez que hacemos esta identificación intuitiva, sin embargo, sólo queda un pequeño paso para decidir ajustar los parámetros $a_1 \dots a_M$ precisamente encontrando aquellos valores que *maximizan* la verosimilitud definida de la manera anterior. Esta forma de estimación de parámetros se denomina *estimación de máxima verosimilitud*.

Ya estamos listos para hacer la conexión con (15.1.2). Supongamos que cada dato y_i tiene un error de medición que es independiente al resto y distribuido normalmente (Gaussiano) alrededor del modelo “verdadero” $y(x)$. Y supongamos que las desviaciones estándar de estas distribuciones normales son las mismas para todos los puntos. Entonces la probabilidad de los datos es el producto de las probabilidades de cada punto:

$$P \propto \prod_{i=1}^N \left[\exp \left(-\frac{1}{2} \left(\frac{y_i - y(x_i)}{\sigma} \right)^2 \right) \Delta y \right] \quad (4.1)$$

Observa que hay un factor Δy en cada término del producto. Maximizar (15.1.3) es equivalente a maximizar su logaritmo, o minimizar el negativo de su logaritmo, a saber,

$$\sum_{i=1}^N \frac{[y_i - y(x_i)]^2}{2\sigma^2} - N \log \Delta y \quad (4.2)$$

Dado que N , σ , y Δy son constantes, minimizar esta expresión es equivalente a minimizar (15.1.2).

Lo que vemos es que el ajuste por mínimos cuadrados es una estimación de máxima verosimilitud de los parámetros ajustados si los errores de medición son independientes y están normalmente distribuidos con desviación estándar constante. Observa que no hicimos ninguna suposición sobre la linealidad o no linealidad del modelo $y(x; a_1 \dots a_M)$ en sus parámetros $a_1 \dots a_M$. Pero si relajamos nuestra suposición de desviación estándar constante y obtenemos las fórmulas similares para lo que se llama “ajuste por chi-cuadrado” o *weighted least-squares fitting*. Primero, sin embargo, discutiremos más a fondo nuestro supuesto de una distribución normal.

La siguiente sección tiene que ver con el enamoramiento de la comunidad científica con la distribución normal. Esta fascinación tiende a alejar la atención del hecho de que, para datos reales, la distribución normal es con frecuencia pobremente realizada, si se realiza en absoluto. Estamos acostumbrados a decir, de forma bastante casual, que, en promedio, las mediciones caerán dentro del 68 % del valor verdadero con un error de $\pm 2\sigma$ con un 95 % de certeza, y dentro de $\pm 3\sigma$ con 99.7 % de certeza. Extendiendo esto, uno esperaría una medición fuera de $\pm 20\sigma$ sólo una vez entre 2×10^{85} . Todos sabemos que los “fallos” son mucho más probables que eso.

En algunos casos, las desviaciones de una distribución normal son fáciles de entender y cuantificar. Por ejemplo, en mediciones obtenidas por conteo de eventos, los errores de medición suelen estar distribuidos según una distribución de Poisson, cuya función de distribución acumulada ya se discutió en la sección 6.2. Cuando el número de cuentas en un solo punto de datos es grande, la distribución de Poisson converge hacia una Gaussiana. Sin embargo, la convergencia no es uniforme cuando se mide en precisión fraccional. Cuanto más desviación estándar haya en la cola de la distribución, mayor será el número de cuentas antes de que se alcance un valor cercano a la distribución Gaussiana. El signo del efecto siempre es el mismo: la Gauss predice que los “eventos extremos” son mucho menos probables de lo que realmente son (según Poisson). Esto hace que tales eventos, cuando ocurren, desvíen un ajuste por mínimos cuadrados mucho más de lo que deberían.

Otras veces, las desviaciones de una distribución normal no son tan fáciles de entender en detalle. Los puntos experimentales a veces simplemente se salen del patrón. Tal vez la lámpara parpadeó durante la medición de un punto, o alguien golpeó el aparato, o alguien escribió mal un número. Puntos como estos se llaman *outliers*. Pueden arruinar fácilmente un ajuste por mínimos cuadrados adecuado. Su probabilidad de ocurrencia en el modelo Gaussiano asumido es tan pequeña que el estimador de máxima verosimilitud tratará de distorsionar toda la curva para tratar de incluirlos, erróneamente.

El tema de los *errores robustos* trata los casos donde el modelo normal o Gaussiano es una mala aproximación, o donde los *outliers* son importantes. Discutiremos brevemente estos métodos en el §15.7. Todas las secciones entre ésta y aquella tratan un caso u otro, una distribución Gaussiana para los errores de medición o no. Es bastante importante que mantengas en mente las limitaciones de ese modelo, incluso cuando lo uses. Los resultados pueden parecer buenos sin serlo.

Finalmente, observa que nuestra discusión de errores de medición se ha limitado a errores *estadísticos*, no a aquellos que promedian al repetir muchas veces el experimento. No se puede eliminar un error sistemático simplemente promediando. Por ejemplo, el calibrado de un medidor de metal puede depender de la temperatura. Si tomamos todas nuestras mediciones en el mismo espacio con la misma temperatura, entonces ningún número de promedios o repeticiones corregirá este error sistemático no reconocido.

4.2.1. Ajuste por Chi-Cuadrada

Ya consideramos la estadística de chi-cuadrada anteriormente, en la §14.3. Aquí surge en un contexto ligeramente diferente.

Si cada punto de datos (x_i, y_i) tiene su propia desviación estándar conocida σ_i , entonces la ecuación (15.1.3) se modifica únicamente colocando un subíndice i en el símbolo σ . Ese subíndice también se propaga dócilmente en (15.1.4), de modo que la estimación de máxima verosimilitud de los parámetros del modelo se obtiene minimizando la cantidad

$$\chi^2 \equiv \sum_{i=1}^N \left[\frac{y_i - y(x_i; a_1 \dots a_M)}{\sigma_i} \right]^2 \quad (15.1.5)$$

llamada “chi-cuadrada.”

En la medida en que los errores de medición realmente estén distribuidos normalmente, la cantidad χ^2 es correspondientemente una suma de N cuadrados de variables normalmente distribuidas, cada una normalizada a varianza unitaria. Una vez que hemos ajustado los $a_1 \dots a_M$ para minimizar el valor de χ^2 , los términos en la suma ya no son todos estadísticamente independientes. Para modelos que son lineales en los a 's, sin embargo, resulta que la distribución de probabilidad para diferentes valores de χ^2 en su mínimo puede derivarse analíticamente, y es la *distribución chi-cuadrada para $N - M$ grados de libertad*. Aprendimos cómo calcular esta función de probabilidad usando la función gamma incompleta discutida en la §6.2. En particular, la ecuación (6.2.18) da la probabilidad Q de que la chi-cuadrada haya excedido un valor particular χ^2 por casualidad, donde $\nu = N - M$ es el *número de grados de libertad*. La cantidad Q , o su complemento $P \equiv 1 - Q$, se cita frecuentemente en los apéndices de los libros de estadística, pero generalmente encontramos más fácil usar **gammp** y calcular nuestros propios valores: $Q = \text{gammp}(0.5\nu, 0.5\chi^2)$. Es bastante común, y usualmente no muy erróneo, asumir que la distribución chi-cuadrada se mantiene incluso para modelos que no son estrictamente lineales en los a 's.

El valor de Q da una medida cuantitativa de la bondad del ajuste del modelo. Si Q es una probabilidad muy pequeña para algún conjunto de datos en particular, entonces el modelo (i) es incorrecto — puede ser

estadísticamente rechazado, o (ii) alguien ha mentido sobre el tamaño de los errores de medición σ_i — están mal informados o subestimados.

Es importante señalar que la probabilidad chi-cuadrada Q no evalúa directamente la credibilidad de la suposición de que los errores de medición están distribuidos normalmente. Lo asume. En la mayoría, pero no en todos los casos, sin embargo, el efecto de errores no normales es crear una abundancia de valores atípicos. Estos disminuyen la probabilidad Q , para que podamos agregar otra posible, aunque menos definitiva, conclusión a la lista anterior: (iii) los errores de medición pueden no estar distribuidos normalmente.

La posibilidad (iii) es bastante común, y también bastante benigna. Es por eso que los procedimientos estándar son a menudo bastante tolerantes con probabilidades bajas Q . No es poco común aceptar en igualdad de condiciones muchos modelos con, digamos, $Q > 0,001$. Pero no se debe ser tan descuidado como los periodistas: Los modelos verdaderamente erróneos serán rechazados con valores mucho más pequeños de Q , digamos 10^{-8} , digamos. Sin embargo, si día tras día descubres que tus conjuntos de datos te están dando valores de $\chi^2 \sim \nu$, realmente deberías rastrear la causa.

Si conoces la distribución real de tus errores de medición, entonces puedes usar métodos de Monte Carlo para *simular* algunos conjuntos de datos desde una distribución particular, §§7.2–§7.3. Luego puedes comparar estos datos sintéticos con los tuyos para determinar si la distribución de probabilidad asumida para la estadística χ^2 describe adecuadamente la precisión con la que se reproducen tus parámetros modelo. Discutiremos esto más a fondo en §15.6. Las estadísticas chi-cuadradas son muy generales, pero debes tener cuidado al usarlas.

Finalmente, ten en cuenta que nuestra discusión de errores de medición ha estado limitada a *errores estadísticos*, no a aquellos cuyo valor promedio se alejará incluso de la media bajo promedios. Por ejemplo, la calibración de un medidor de metal puede depender de su temperatura. Si tomamos todos nuestros valores de medición en la misma temperatura, entonces ningún monto de promediar o regresión corregirá este error *sistemático no reconocido*.

Una regla práctica es que un valor “típico” de χ^2 para un ajuste “moderadamente” bueno es $\chi^2 \approx \nu$. Más precisamente, la afirmación es que la estadística χ^2 tiene una media ν y una desviación estándar $\sqrt{2\nu}$, y, asintóticamente para grandes ν , se convierte en una distribución normal.

A veces los científicos usan conjuntos de datos con un conjunto de errores de medición que no se conocen de antemano, y las consideraciones relacionadas con el ajuste de χ^2 se usan para derivar un valor de σ . Si asumimos que todas las mediciones tienen la misma desviación estándar, $\sigma_i = \sigma$, y que el modelo se ajusta bien, entonces podemos proceder primero asignando un valor arbitrario a σ y a todos los puntos, luego ajustar los parámetros del modelo minimizando χ^2 , y finalmente recalculando

$$\sigma^2 = \sum_{i=1}^N [y_i - y(x_i)]^2 / (N - M) \quad (15.1.6)$$

Obviamente, este enfoque prohíbe una evaluación independiente de la bondad del ajuste, un hecho ocasionalmente pasado por alto por sus seguidores. Sin embargo, cuando el error de medición no se conoce, este enfoque permite al menos que algún tipo de barra de error sea asignada a los puntos.

Si tomamos la derivada de la ecuación (15.1.5) con respecto a los parámetros a_k , obtenemos ecuaciones que deben cumplirse en el mínimo chi-cuadrado,

$$0 = \sum_{i=1}^N \frac{[y_i - y(x_i)]}{\sigma_i^2} \left(\frac{\partial y(x_i; \dots a_k \dots)}{\partial a_k} \right), \quad k = 1, \dots, M \quad (15.1.7)$$

La ecuación (15.1.7) es, en general, un conjunto de M ecuaciones no lineales para los M parámetros desconocidos a_k . Varias de los procedimientos descritos en este capítulo derivan de (15.1.7) y sus especializaciones.

4.3. Mínimos Cuadrados Lineales Generales

Una generalización inmediata de la §15.2 es ajustar un conjunto de puntos de datos (x_i, y_i) a un modelo que no sea solo una combinación lineal de 1 y x (es decir, $a + bx$), sino más bien una combinación lineal de cualquier conjunto M de funciones especificadas de x . Por ejemplo, las funciones podrían ser $1, x, x^2, \dots, x^{M-1}$, en cuyo caso su combinación lineal general,

$$y(x) = a_1 + a_2x + a_3x^2 + \dots + a_Mx^{M-1} \quad (15.4.1)$$

es un polinomio de grado $M-1$. O bien, las funciones podrían ser senos y cosenos, en cuyo caso su combinación lineal general es una serie armónica.

La forma general de este tipo de modelo es

$$y(x) = \sum_{k=1}^M a_k X_k(x) \quad (15.4.2)$$

donde $X_1(x), \dots, X_M(x)$ son funciones fijas arbitrarias de x , llamadas *funciones base*.

Observa que las funciones $X_k(x)$ pueden ser funciones altamente no lineales de x . En esta discusión, “lineal” se refiere únicamente a la dependencia del modelo con respecto a sus *parámetros* a_k .

Para estos modelos lineales generalizamos la discusión de la sección anterior definiendo una función de mérito

$$\chi^2 = \sum_{i=1}^N \left[\frac{y_i - \sum_{k=1}^M a_k X_k(x_i)}{\sigma_i} \right]^2 \quad (15.4.3)$$

Como antes, σ_i es el error de medición (desviación estándar) del punto de datos i -ésimo, que se presume conocido. Si los errores de medición no son conocidos, pueden (como ya se discutió al final de la §15.1) establecerse en el valor constante $\sigma = 1$.

Una vez más, elegiremos como mejores parámetros aquellos que minimicen χ^2 . Existen varias técnicas diferentes disponibles para encontrar este mínimo. Dos son particularmente útiles, y discutiremos ambas en esta sección. Para introducirlas y aclarar su relación, necesitaremos algo de notación.

Sea $\mathbf{A}$ una matriz cuyos componentes $N \times M$ son construidos a partir de las M funciones base evaluadas en las N abscisas x_i , y de los N errores de medición σ_i , por la prescripción

$$A_{ij} = \frac{X_j(x_i)}{\sigma_i} \quad (15.4.4)$$

La matriz $\mathbf{A}$ se llama *matriz de diseño* del problema de ajuste. Observa que en general $\mathbf{A}$ tiene más filas que columnas, $N \geq M$, ya que debe haber más puntos de datos que parámetros del modelo a resolver. (¡Puedes ajustar una línea recta a dos puntos, pero no una quintica muy significativa!) La matriz de diseño se muestra esquemáticamente en la Figura 15.4.1.

También definimos un vector $\mathbf{b}$ de longitud N mediante

$$b_i = \frac{y_i}{\sigma_i} \quad (15.4.5)$$

y denotamos al vector M cuyos componentes son los parámetros a ajustar, $a_1, \dots, a_M$, como $\mathbf{a}$.

4.3.1. Solución mediante el uso de las Ecuaciones Normales

El mínimo de (15.4.3) ocurre donde la derivada de χ^2 con respecto a todos los parámetros a_k se anula. Especializando la ecuación (15.1.7) al caso del modelo (15.4.2), esta condición produce las M ecuaciones

$$0 = \sum_{i=1}^N \frac{1}{\sigma_i^2} \left[y_i - \sum_{j=1}^M a_j X_j(x_i) \right] X_k(x_i), \quad k = 1, \dots, M \quad (4.3)$$

Intercambiando el orden de las sumas, podemos escribir (15.4.6) como la ecuación matricial

$$\sum_{j=1}^M \alpha_{kj} a_j = \beta_k \quad (4.4)$$

donde

$$\alpha_{kj} = \sum_{i=1}^N \frac{X_j(x_i) X_k(x_i)}{\sigma_i^2}, \quad \text{o equivalentemente} \quad [\alpha] = \mathbf{A}^T \cdot \mathbf{A} \quad (4.5)$$

una matriz $M \times M$, y

$$\beta_k = \sum_{i=1}^N \frac{y_i X_k(x_i)}{\sigma_i^2}, \quad \text{o equivalentemente} \quad [\beta] = \mathbf{A}^T \cdot \mathbf{b} \quad (4.6)$$

un vector de longitud M .

Las ecuaciones (15.4.6) o (15.4.7) se llaman las *ecuaciones normales* del problema de mínimos cuadrados. Pueden resolverse para el vector de parámetros $\mathbf{a}$ por los métodos estándar del Capítulo 2, ya sea descomposición LU , eliminación hacia atrás y hacia adelante, descomposición de Cholesky, o eliminación de Gauss-Jordan. En forma matricial, las ecuaciones normales pueden escribirse como

$$[\alpha] \cdot \mathbf{a} = [\beta] \quad \text{o} \quad (\mathbf{A}^T \cdot \mathbf{A}) \cdot \mathbf{a} = \mathbf{A}^T \cdot \mathbf{b} \quad (4.7)$$

La matriz inversa $C_{jk} \equiv [\alpha]_{jk}^{-1}$ está estrechamente relacionada con las incertidumbres *probables* (o, más precisamente, *estándar*) de los parámetros estimados. Para estimar estas incertidumbres, consideramos que

$$a_j = \sum_{k=1}^M [\alpha]_{jk}^{-1} \beta_k = \sum_{k=1}^M C_{jk} \left[\sum_{i=1}^N \frac{y_i X_k(x_i)}{\sigma_i^2} \right] \quad (4.8)$$

y que la varianza asociada con la estimación a_j puede encontrarse como en (15.2.7) a partir de

$$\sigma^2(a_j) = \sum_{i=1}^N \sigma_i^2 \left(\frac{\partial a_j}{\partial y_i} \right)^2 \quad (4.9)$$

Note que a_j es independiente de y_i , por lo tanto

$$\frac{\partial a_j}{\partial y_i} = \sum_{k=1}^M C_{jk} X_k(x_i) / \sigma_i^2 \quad (4.10)$$

En consecuencia, obtenemos que

$$\sigma^2(a_j) = \sum_{k=1}^M \sum_{l=1}^M C_{jk} C_{jl} \left[\sum_{i=1}^N \frac{X_k(x_i) X_l(x_i)}{\sigma_i^2} \right] \quad (4.11)$$

El término final entre corchetes es simplemente la matriz $[\alpha]$. Como esta es la inversa de $[C]$, (15.4.14) se reduce inmediatamente a

$$\sigma^2(a_j) = C_{jj} \quad (4.12)$$

En otras palabras, los elementos diagonales de $[C]$ son las varianzas (incertidumbres cuadradas) de los parámetros ajustados. No debería sorprender que los elementos fuera de la diagonal C_{jk} sean las covarianzas entre a_j y a_k (cf. 15.2.10); pero dejaremos la discusión de esto hasta el §15.6.

Implementaremos una rutina que utiliza las fórmulas anteriores para el problema general de mínimos cuadrados lineales, y no solo resuelve para la solución vectorial $\mathbf{a}$ sino también para la matriz de covarianzas $[C]$, es más eficiente utilizar la eliminación de Gauss-Jordan (cf. §2.1) para realizar el álgebra lineal. La operación requerida, en esta aplicación, no es mayor que para la descomposición LU .

Si sabes de antemano que algunos de los parámetros a_k deben mantenerse fijos en ciertos valores debido a teoría previa o mediciones anteriores, puedes congelarlos (ponerlos constantes). En la rutina siguiente, se proporciona un arreglo `ia[1..ma]`, cuyos componentes son cero o no cero (*e.g.*, 1). Ceros indican que el valor correspondiente del vector de parámetros `a[1..ma]` debe mantenerse fijo. En la salida, los parámetros fijados no tendrán varianza y todas sus covarianzas se establecerán en cero en la matriz de covarianzas.


```

1 #include "nrutil.h"
2
3 void lfit(float x[], float y[], float sig[], int ndat, float a[], int ia[],
4 int ma, float **covar, float *chisq, void (*funcs)(float, float [], int))
5 /* Dado un conjunto de puntos de datos x[1..ndat], y[1..ndat] con desviaciones estándar
6 individuales sig[1..ndat], usa minimización de  $\chi^2$  para ajustar algunos o todos los
7 coeficientes a[1..ma]
8 de una función que depende linealmente de a,  $y = \sum a_i \times \text{afuncs}(x)$ . El arreglo de
9 entrada ia[1..ma]
10 indica mediante valores distintos de cero aquellos componentes de a que deben ser
11 ajustados, y mediante ceros
12 aquellos componentes que deben mantenerse fijos en sus valores de entrada. El programa
13 devuelve los valores
14 para a[1..ma],  $\chi^2 = \text{chisq}$ , y la matriz de covarianza covar[1..ma][1..ma]. (Parámetros
15 mantenidos fijos retornarán covarianzas cero.) El usuario suministra una rutina funcs(
16 x,afunc,ma) que
17 devuelve las ma funciones base evaluadas en x = x en el arreglo afunc[1..ma].
18 */
19 {
20     void covsrt(float **covar, int ma, int ia[], int mfit);
21     void gaussj(float **a, int n, float **b, int m);
22     int i,j,k,l,m,mfit=0;
23     float ym,wt,sum,sig2i,**beta,**afunc;
24
25     beta=matrix(1,ma,1,1);
26     afunc=vector(1,ma);
27     for (j=1;j<=ma;j++)
28         if (ia[j]) mfit++;
29     if (mfit == 0) nrerror("lfit: no parameters to be fitted");
30     for (j=1;j<=mfit;j++) { /* Inicializa la matriz (simétrica). */
31         for (k=1;k<=mfit;k++) covar[j][k]=0.0;
32         beta[j][1]=0.0;
33     }
34
35     for (i=1;i<=ndat;i++) { /* Recorre los datos para acumular coeficientes
36                             de las ecuaciones normales. */
37         (*funcs)(x[i],afunc,ma);
38         ym=y[i];
39         if (mfit < ma) {
40             for (j=1;j<=ma;j++)
41                 if (!ia[j]) ym -= a[j]*afunc[j]; /* Sustrae dependencias de las partes
42                 conocidas de la función de ajuste. */
43         }
44         sig2i=1.0/SQR(sig[i]);
45         for (j=0,l=1;l<=ma;l++) {
46             if (ia[l]) {
47                 wt=afunc[l]*sig2i;
48                 for (j++,k=0,m=1;m<=l;m++)
49                     if (ia[m]) covar[j][++k] += wt*afunc[m];
50                 beta[j][1] += ym*wt;
51             }
52         }
53     }
54
55     for (j=2;j<=mfit;j++)
56         for (k=1;k<j;k++) covar[k][j]=covar[j][k]; /* Completa por simetría. */
57     gaussj(covar,mfit,beta); /* Solución matricial. */
58     for (j=0,l=1;l<=ma;l++)
59         if (ia[l]) a[l]=beta[l][1]; /* Asigna solución a los coeficientes apropiados a
60         . */
61     *chisq=0.0;
62     for (i=1;i<=ndat;i++) {
63         (*funcs)(x[i],afunc,ma);
64         for (sum=0.0,j=1;j<=ma;j++) sum += a[j]*afunc[j];
65         *chisq += SQR((y[i]-sum)/sig[i]); /* Evalúa  $\chi^2$  del ajuste. */
66     }
67 }

```

```

59     }
60     covsrt(covar,ma,ia,mfit);          /* Ordena matriz de covarianza a verdadero orden de
        los
61                                     coeficientes de ajuste. */
62     free_vector(afunc,1,ma);
63     free_matrix(beta,1,ma,1,1);
64 }

```

4.3.2. Solución mediante el uso de la descomposición en valores singulares

En algunas aplicaciones, las ecuaciones normales son perfectamente adecuadas para problemas de mínimos cuadrados lineales. Sin embargo, en muchos casos, las ecuaciones normales son muy cercanas a ser singulares. Un elemento pivote cero puede encontrarse durante la solución de las ecuaciones lineales (por ejemplo, en `gaussj`), en cuyo caso no se obtiene solución en absoluto. O puede encontrarse un pivote muy pequeño, en cuyo caso típicamente se obtienen parámetros ajustados a_k con magnitudes muy grandes que están (delicada y exactamente) balanceadas para cancelarse casi precisamente cuando se evalúa la función ajustada.

¿Por qué ocurre esto comúnmente? La razón es que, más a menudo de lo que los experimentadores quisieran admitir, los datos no distinguen claramente entre dos o más de las funciones base provistas. Si dos tales funciones, o dos combinaciones diferentes de funciones, resultan ajustar los datos igualmente bien — o igualmente mal — entonces la matriz $[\alpha]$, incapaz de distinguir entre ellas, colapsa y se vuelve singular. Hay cierta ironía matemática en el hecho de que los problemas de mínimos cuadrados son tanto *sobredeterminados* (número de puntos de datos mayor que número de parámetros) como *subdeterminados* (existen combinaciones ambiguas de parámetros), pero esto ocurre frecuentemente. Las ambigüedades pueden ser extremadamente difíciles de notar *a priori* en problemas complicados.

Entra en escena la descomposición en valores singulares (SVD). Este sería un buen momento para revisar lo explicado en la sección §2.6, que no repetiremos aquí. En el caso de un sistema sobredeterminado, SVD produce una solución que es la mejor aproximación en el sentido de los mínimos cuadrados, cf. ecuación (2.6.10). Eso es exactamente lo que queremos. En el caso de un sistema subdeterminado, SVD produce una solución cuyos valores (para los a_k) son mínimos en el sentido de los mínimos cuadrados, cf. ecuación (2.6.8). Eso también es lo que queremos: cuando alguna combinación de funciones base es irrelevante para el ajuste, esa combinación será reducida a un valor pequeño, inocuo, en lugar de empujada hacia infinitos delicadamente cancelantes.

En términos de la matriz de diseño $\mathbf{A}$ (ecuación 15.4.4) y del vector $\mathbf{b}$ (ecuación 15.4.5), la minimización de χ^2 en (15.4.3) puede escribirse como

$$\text{Encontrar } \mathbf{a} \text{ que minimice } \chi^2 = |\mathbf{A} \cdot \mathbf{a} - \mathbf{b}|^2 \quad (15.4.16)$$

Comparando con la ecuación (2.6.9), vemos que este es precisamente el problema que las rutinas `svdcmp` y `svbksb` están diseñadas para resolver. La solución, dada por la ecuación (2.6.12), puede reescribirse como sigue: si $\mathbf{U}$ y $\mathbf{V}$ ingresan a la descomposición SVD de acuerdo con la ecuación (2.6.13), computada por `svdcmp`, entonces los vectores $\mathbf{U}_{(i)}$ con $i = 1, \dots, M$ son las columnas de $\mathbf{U}$ (cada una un vector de longitud N); y los vectores $\mathbf{V}_{(j)}$ con $j = 1, \dots, M$ denotan las columnas de $\mathbf{V}$ (cada una un vector de longitud M). Entonces, la solución de mínimos cuadrados (15.4.16) puede escribirse como

$$\mathbf{a} = \sum_{i=1}^M \frac{(\mathbf{U}_{(i)} \cdot \mathbf{b})}{w_i} \mathbf{V}_{(i)} \quad (15.4.17)$$

Aunque está más allá de nuestro alcance probarlo aquí, resulta que los errores estándar (locales, "probables") en los parámetros ajustados también son combinaciones lineales de las columnas de $\mathbf{V}$. De hecho, la ecuación (15.4.17) puede escribirse en una forma que muestra estos errores como

$$\mathbf{a} = \left[\sum_{i=1}^M \frac{(\mathbf{U}_{(i)} \cdot \mathbf{b})}{w_i} \mathbf{V}_{(i)} \right] \pm \left[\frac{1}{w_1} \mathbf{V}_{(1)} \pm \dots \pm \frac{1}{w_M} \mathbf{V}_{(M)} \right] \quad (15.4.18)$$

Aquí, el signo $\pm$ es seguido por una desviación estándar. Lo sorprendente es que, descompuestos de esta manera, las desviaciones estándar son todas mutuamente independientes (no correlacionadas). Por lo tanto, pueden sumarse en cuadratura. Lo que ocurre es que los vectores $\mathbf{V}_{(i)}$ son los ejes principales del elipsoide de error de los parámetros ajustados (ver §15.6).

Se sigue que la varianza en la estimación de un parámetro a_j viene dada por

$$\sigma^2(a_j) = \sum_{i=1}^M \frac{1}{w_i^2} V_{(i)j}^2 = \sum_{i=1}^M \left(\frac{V_{ij}}{w_i} \right)^2 \quad (15.4.19)$$

cuyo resultado debe coincidir con (15.4.14). Como antes, no debes sorprenderte de que la fórmula para las covarianzas, aquí dada sin demostración, sea

$$\text{Cov}(a_j, a_k) = \sum_{i=1}^M \left(\frac{V_{ij}V_{ik}}{w_i^2} \right) \quad (15.4.20)$$

Hemos introducido esta subsección notando que las ecuaciones normales pueden fallar al encontrar un pivote cero. Sin embargo, no hemos notado aún cómo SVD supera este problema. La respuesta es: Si algún valor singular w_i es cero, su recíproco en la ecuación (15.4.18) debe establecerse en cero, no en infinito. (Comparar con la discusión anterior a la ecuación 2.6.7.) Esto corresponde a agregar al conjunto de parámetros ajustados un múltiplo cero, más bien que algún múltiplo aleatorio grande, de cualquier combinación lineal de funciones base que sea degenerada en el ajuste. ¡Es una buena cosa por hacer!

Además, si un valor singular w_i no es cero pero muy pequeño, también deberías establecer su recíproco en cero, ya que su valor aparente probablemente es un artefacto de error numérico, no una cantidad significativa. Una respuesta plausible a la pregunta "¿qué tan pequeño es pequeño?" es editar a mano todos los valores singulares cuyo cociente respecto al mayor valor singular sea menor que N veces la precisión de la máquina ϵ . (Podrías multiplicar por $\sqrt{N}$, o una constante, en lugar de N como el multiplicador; eso comienza a adentrarse en cuestiones específicas del hardware y del problema.)

Hay otra razón para editar manualmente valores singulares pequeños: aquellos lo son precisamente porque el número no es una pregunta. La descomposición en valores singulares siempre permite combinaciones lineales de variables que simplemente no contribuyen mucho a reducir la función χ^2 para tu conjunto de datos. Editarlas puede reducir el número de grados de libertad efectivos bastante significativamente, mientras que incrementa poco la función χ^2 . Nos gustaría saber más sobre cómo identificar y tratar estos casos de forma automática. Si sabes cómo hacer esto, ¡dínoslo! Mientras tanto, la edición manual resultará ser una técnica bastante poderosa.

Un problema menor es que la SVD requiere almacenamiento adicional para una matriz de tamaño $N \times M$ para almacenar la matriz de diseño completa. Este almacenamiento es sobrescrito por la matriz $\mathbf{U}$. También se requiere almacenamiento para la matriz $\mathbf{V}$ de tamaño $M \times M$, pero esto es en lugar de la matriz del mismo tamaño para los coeficientes de las ecuaciones normales. La SVD puede ser significativamente más lenta que resolver las ecuaciones normales; sin embargo, su gran ventaja, que (teóricamente) *no puede fallar*, compensa más que de sobra la desventaja en velocidad.

En la rutina que sigue, las matrices $\mathbf{u}$, $\mathbf{v}$ y el vector $\mathbf{w}$ se ingresan como espacio de trabajo. Las dimensiones lógicas del problema son `ndat` puntos de datos por `ma` funciones base (y parámetros ajustados). Si solo te importan los valores de $\mathbf{a}$ de los parámetros ajustados, entonces $\mathbf{u}$, $\mathbf{v}$ y $\mathbf{w}$ no contienen información útil en la salida. Si deseas errores probables para los parámetros ajustados, continúa leyendo.

Listing 4.1: Ajuste por descomposición en valores singulares

```

1 #include "nrutil.h"
2 #define TOL 1.0e-5 // Valor por defecto para precisión simple y variables escaladas a la
   unidad
3
4 void svdfit(float x[], float y[], float sig[], int ndat, float a[], int ma,
5           float **u, float **v, float w[], float *chisq,
6           void (*funcs)(float, float [], int))
7 {
8     void svbksb(float **u, float w[], float **v, int m, int n, float b[], float x[]);
9     void svdcmp(float **a, int m, int n, float w[], float **v);
10    int i,j;
11    float wmax,tmp,thresh,sum,*b,*afunc;
12
13    b=vector(1,ndat);
14    afunc=vector(1,ma);
15    for (i=1;i<=ndat;i++) { // Acumula los coeficientes de la matriz del ajuste.
16        (*funcs)(x[i],afunc,ma);
17        tmp=1.0/sig[i];
18        for (j=1;j<=ma;j++) u[i][j]=afunc[j]*tmp;
19        b[i]=y[i]*tmp;
20    }
21
22    svdcmp(u,ndat,ma,w,v); // Descomposición en valores singulares
23    wmax=0.0;
24    for (j=1;j<=ma;j++) // Edita los valores singulares, con TOL desde la definición
25        if (w[j] > wmax) wmax=w[j];
26    thresh=TOL*wmax;

```

```

27     for (j=1;j<=ma;j++)
28         if (w[j] < thresh) w[j]=0.0;
29     svbksb(u,w,v,ndat,ma,b,a); // Evalúa chi-cuadrado.
30     *chisq=0.0;
31     for (i=1;i<=ndat;i++) {
32         (*funcs)(x[i],afunc,ma);
33         for (sum=0.0,j=1;j<=ma;j++) sum += a[j]*afunc[j];
34         *chisq += (tmp=(y[i]-sum)/sig[i],tmp*tmp);
35     }
36     free_vector(afunc,1,ma);
37     free_vector(b,1,ndat);
38 }

```

Al introducir la matriz v y el vector w producidos por el programa anterior en la siguiente rutina breve, se obtienen fácilmente las varianzas y covarianzas de los parámetros ajustados a . Las raíces cuadradas de las varianzas son las desviaciones estándar de los parámetros ajustados. La rutina implementa directamente la ecuación (15.4.20) anterior, con la convención de que los valores singulares iguales a cero son reconocidos como eliminados del ajuste.

Listing 4.2: Cálculo de la matriz de covarianza con descomposición SVD

```

1  #include "nrutil.h"
2
3  void svdvar(float **v, int ma, float w[], float **cvm)
4  // Evalúa la matriz de covarianza cvm[1..ma][1..ma] del ajuste para los parámetros
   obtenidos por svdfit.
5  // Llama a esta rutina con las matrices v[1..ma][1..ma] y w[1..ma] tal como se obtienen
   en svdfit.
6  {
7      int k,j,i;
8      float sum,*wti;
9
10     wti=vector(1,ma);
11     for (i=1;i<=ma;i++) {
12         wti[i]=0.0;
13         if (w[i]) wti[i]=1.0/(w[i]*w[i]);
14     }
15
16     for (i=1;i<=ma;i++) { // Suma contribuciones a la matriz de covarianza (15.4.20).
17         for (j=1;j<=i;j++) {
18             for (sum=0.0,k=1;k<=ma;k++) sum += v[i][k]*v[j][k]*wti[k];
19             cvm[j][i]=cvm[i][j]=sum;
20         }
21     }
22     free_vector(wti,1,ma);
23 }

```